

Review

Towards efficient language giants: A comprehensive survey on structural optimizations and compression techniques for large language models

Gilhyeon Lee¹, Seonggeun Kim¹, Dongjun Lee, Kyungmin Goh, Hyun Kim^{ID*}*Department of Electrical and Information Engineering, Research Center for Electrical and Information Technology, Seoul National University of Science and Technology, 232 Gongneung-ro, Nowon-gu, Seoul, 01811, Republic of Korea*

ARTICLE INFO

Keywords:

Large language models
Efficient inference
Model-level optimization
Survey

ABSTRACT

The impressive success of large language models (LLMs) across a broad spectrum of NLP tasks has attracted considerable attention in both academia and industry. However, their inference incurs substantial computational and memory overhead, making it challenging to efficiently deploy them in latency-sensitive or resource-constrained environments. This survey provides a comprehensive overview of model-level optimization techniques developed to address these issues. Specifically, we examine efficient module designs for core Transformer components—including multi-head attention, feed-forward networks, normalization, and positional encoding—and summarize representative compression methods such as quantization, pruning, knowledge distillation, approximation, and parameter sharing. We categorize recent advances in each area, analyzing their algorithmic principles, empirical trade-offs, and impact on inference efficiency. Furthermore, we propose practical design strategies that effectively combine efficient modules with compression techniques to significantly reduce inference costs while meeting application-specific accuracy and latency requirements.

1. Introduction

Recent advancements in large-scale language models (LLMs), including GPT (Brown et al., 2020) and LLaMA (Touvron et al., 2023a), as well as the emergence of various open-source models such as OPT (Zhang et al., 2022a), Vicuna (Vicuna, 2023), BLOOM (Le Scao et al., 2023), Falcon (Almazrouei et al., 2023), and Qwen (Bai et al., 2023a), have considerably facilitated the development and deployment of models with millions to hundreds of billions of parameters across academic and industrial sectors. However, these performance gains inherently lead to substantial increases in memory consumption and computational demands during inference. These factors become critical bottlenecks in real-world deployments, particularly in scenarios that require rapid, low-latency responses. Specifically, Transformer-based models generally require approximately $2 \cdot n \cdot p$ floating-point operations (FLOPs) to generate n output tokens with p parameters. Additionally, these models consume significant memory; under FP16 precision, each billion parameters typically demands around 2 GB of GPU memory. As the model size grows, latency due to memory access increasingly contributes to delays (Barros, 2025). For example, GPT3-175B (Brown et al., 2020) exhibits a per-token latency exceeding 350 ms on a single NVIDIA A100

GPU, making it challenging to achieve the response times typically expected in real-time systems, which are below 100–200 ms (Barros, 2025). These limitations are particularly acute in environments with constrained computational resources, such as edge devices, and in server deployments that handle high volumes of simultaneous inference requests.

To overcome these limitations, inference optimization research has been actively pursued and can be specifically categorized into three domains: system-level, data-level, and model-level optimizations (Wan et al., 2024). System-level optimizations, which do not require additional retraining processes, include memory offloading, kernel optimizations (e.g., FlashAttention Dao, 2024; Dao et al., 2022, xFormers Lefaudeux et al., 2022), pipeline parallelism, and vLLM-based server architecture improvements. Data-level optimizations aim to decrease computational requirements by reducing the length of input prompts or enabling parallel output generation methods such as Skeleton-of-Thought (Ning et al., 2024), thereby minimizing sequential processing overhead. However, these approaches have limitations in substantially reducing FLOPs and memory usage as they do not reduce the model structure or parameters themselves (Zhou et al., 2024). Consequently, model-level optimizations, which directly reduce model structures and

* Corresponding author.

E-mail addresses: gilhyeonlee@seoultech.ac.kr (G. Lee), seonggeunkim@seoultech.ac.kr (S. Kim), dj.lee@seoultech.ac.kr (D. Lee), km23@seoultech.ac.kr (K. Goh), hyunkim@seoultech.ac.kr (H. Kim).¹ Gilhyeon Lee and Seonggeun Kim contributed equally to this work.

parameters, have emerged as more effective and fundamental solutions. Representative approaches include efficient module design to replace transformer modules and compression techniques such as quantization, pruning, knowledge distillation, approximation, and parameter sharing. However, these approaches still face challenges, such as their dependence on specific architectures and the lack of clear guidelines for optimally combining different techniques to balance performance. Moreover, existing surveys tend either to provide broad overviews of the LLM ecosystem (Bai et al., 2024a; Park et al., 2024; Wan et al., 2024; Zhou et al., 2024) or to concentrate on specific lightweight techniques (Cheng et al., 2024; Lang et al., 2024; Xu et al., 2024a). As a result, there remains a relative scarcity of research comprehensively analyzing both module-level modifications and various compression techniques, as well as presenting practical strategies applicable to real-world model designs.

This paper examines various structural modifications and compression techniques proposed for the core modules of LLMs—namely, Multi-head Attention (MHA), Feed-Forward Network (FFN), normalization, and positional embedding (PE). Through this analysis, we outline future directions for model-level optimization and discuss design strategies for constructing practical, efficient models by combining these techniques. The main contributions of this paper are as follows:

- We systematically organize and classify structural modifications of core modules (MHA, FFN, normalization, and PE) to provide a coherent and comprehensive overview.
- We categorize diverse compression techniques into five groups—quantization, pruning, knowledge distillation, approximation, and parameter sharing—and provide comparative insights by analyzing representative studies on quantization, pruning, and knowledge distillation.
- We provide comprehensive and detailed tables that separately summarize and compare existing studies on structural modifications and compression techniques, thereby aiding readers in selecting appropriate modules and compression methods.
- We propose suitable strategies for designing optimized LLMs tailored to different application scenarios, based on the interactions between specific structural modifications and compression techniques.

2. Fundamentals

2.1. Transformer-based LLMs

Fig. 1 illustrates the standard encoder-decoder Transformer architecture (often referred to as the vanilla Transformer) that underpins modern LLMs (Vaswani et al., 2017). The encoder, repeated N times as shown at the bottom of the figure, first combines token embeddings with PEs as inputs, and then alternately stacks MHA and FFN layers. Each MHA and FFN layer is followed by a residual connection (Residual) and a normalization (Norm) module to promote stable training. Similarly, the decoder illustrated at the top follows the same structural pattern, beginning with masked self-attention to ensure tokens do not attend to future positions in the sequence. Subsequently, the decoder employs cross-attention layers to reference the encoder outputs. After passing through N stacked decoder layers, the decoder outputs are projected onto vocabulary logits using a shared linear projection, and these logits are converted into probability distributions via a final softmax operation. The majority of computational costs and memory usage within this architecture are concentrated in four reusable core modules: PE, Normalization, MHA, and FFN.

2.1.1. Positional embedding

Transformer processes all input tokens in parallel, eliminating the sequential bottleneck of recurrent neural network (RNN); however, this architecture alone cannot capture word order information, potentially leading to semantic and grammatical information loss. To address this limitation, vanilla transformer injects positional information by adding

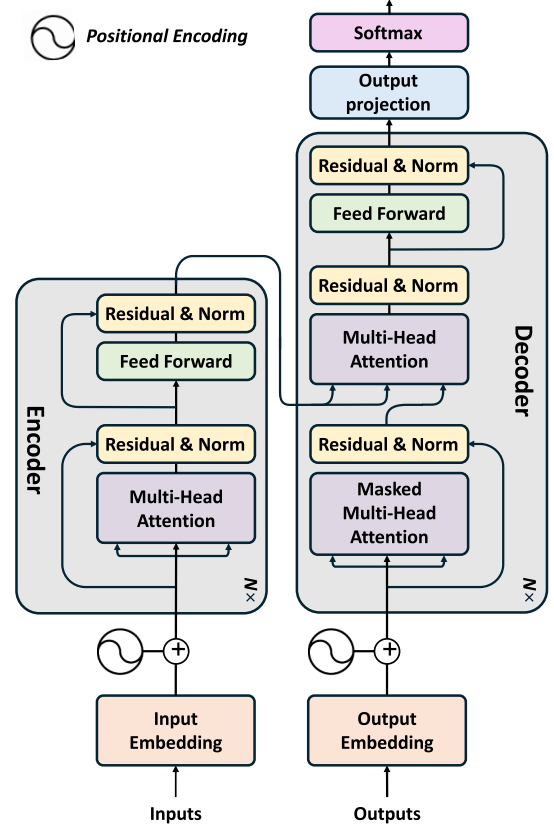


Fig. 1. The transformer architecture proposed in Vaswani et al. (2017), featuring $N \times$ stacked encoder and decoder layers.

fixed sinusoidal PE to each token embedding. PE alternately applies sin and cos waveforms based on token position pos and embedding dimension index i , as expressed follows:

$$\begin{aligned} PE_{(pos, 2i)} &= \sin(pos/10000^{2i/d_{\text{model}}}) \\ PE_{(pos, 2i+1)} &= \cos(pos/10000^{2i/d_{\text{model}}}) \end{aligned} \quad (1)$$

where d_{model} denotes the model embedding dimensionality and the denominator term $10000^{2i/d_{\text{model}}}$ exponentially increases the waveform period corresponding to dimension i , enabling representation of both short-range and long-range information. The variable pos represents the zero-indexed token position, while i denotes the zero-indexed dimension index, with sin applied to even dimensions $2i$ and cos applied to odd dimensions $2i + 1$. By adding these position-specific PE vectors to the original embeddings, self-attention can recognize sequential order and correctly comprehend context.

2.1.2. Normalization

Training instability is one of the major challenges in LLM pretraining. As model size increases, gradients tend to explode or vanish during training. To mitigate this issue, normalization techniques have been widely adopted, helping to accelerate training speed and improve generalization performance by regulating the input or output distributions of each layer. Vanilla transformer adopts LayerNorm (Ba et al., 2016a) as its core normalization method, which applies normalization after residual connections to the outputs of internal sub-modules (e.g., MHA, FFN).

2.1.3. Multi-head attention

MHA is a core component that dynamically determines how much information each token should attend to from others by learning pairwise token dependencies in parallel across the input sequence. MHA divides the attention mechanism into multiple heads, where each head captures

interactions in a different subspace. This allows the model to extract richer contextual information and capture fine-grained relational patterns more effectively than single-head attention (Vaswani et al., 2017; Voita et al., 2019).

1. Generate Query (Q), Key (K), and Value (V) vectors from the input token embeddings.
2. Split Q , K , and V into multiple heads, and compute attention scores for each head by taking the dot product of the corresponding Q and K vectors.
3. Scale the attention scores by the square root of the embedding dimension of K , and apply the softmax function to obtain the attention probabilities.
4. Compute the attention output for each head as the weighted sum of V using the attention probabilities.
5. Concatenate the outputs from all heads and apply a final linear projection to produce the integrated output.

The process begins by generating Q , K , and V from the input embedding X via linear projections, as shown in Eq. (2). Scaled dot-product attention (SDA) is then applied according to Eq. (3), and finally, the outputs from all heads are concatenated and projected to form the final MHA output, as defined in Eq. (4).

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V \quad (2)$$

$$\text{SDA}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (3)$$

$$\text{MHA} = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

$$\text{where } \text{head}_i = \text{SDA}(Q_i, K_i, V_i) \quad (4)$$

2.1.4. Feed-forward network

In Transformer-based LLMs, FFN applies a non-linear transformation to each token representation, enhancing the model's expressiveness. Since FFNs operate independently across tokens, they are inherently parallelizable. An FFN typically consists of two linear transformations with an activation function in between, and can be formulated as follows:

$$y = (\sigma(xW_1 + b_1))W_2 + b_2, \quad (5)$$

where x denotes the input token embedding, W_1, W_2 are weight matrices, b_1, b_2 are bias terms, and $\sigma(\cdot)$ is a non-linear activation function such as ReLU or GELU. The first linear layer typically expands the input into a high-dimensional intermediate representation to capture complex patterns. However, this expansion significantly increases the number of parameters and computation, making the FFN one of the most resource-intensive components in the model. To mitigate this overhead, various lightweight approaches—such as reducing the depth of the FFN or applying pruning—have been actively explored.

Finally, the decoder's hidden states pass through a shared output projection—often weight-tied to the input embedding matrix—to produce vocabulary-sized logits. A softmax then normalizes these logits into probabilities, yielding the language-model (LM) head that drives next-token prediction.

2.2. Architecture types

The original Transformer employs self-attention as its core mechanism and was initially designed as a sequence-to-sequence architecture, where the encoder and decoder interact through cross-attention. Over time, the Transformer architecture has evolved and become specialized to accommodate task-specific requirements and larger model scales. Currently, Transformer architectures are commonly classified into three main categories: (1) Encoder-Only, (2) Decoder-Only, and (3) Encoder-Decoder.

The Encoder-Only architecture leverages bidirectional self-attention, allowing every token in the input sequence to simultaneously reference all other tokens. This characteristic facilitates comprehensive contextual understanding, resulting in superior performance in language understanding tasks such as sentence classification, document embedding, and question answering. Representative models of this type include BERT (Devlin et al., 2019), RoBERTa (Liu et al., 2019), and DeBERTa (He et al., 2020). Specifically, BERT-based models prepend a special classification token [CLS] to the input sequence to extract sentence-level representations, using the hidden state corresponding to this token as a vector representation of the entire sentence. These models have demonstrated robust performance across a variety of natural language understanding benchmarks.

The Decoder-Only architecture utilizes unidirectional self-attention, where each token attends only to itself and preceding tokens through masked self-attention. This design facilitates autoregressive generation, sequentially predicting one token at a time based on all previously generated tokens. Sequence generation typically begins with a special start-of-sequence token (e.g., [BOS] or [S]). Due to these autoregressive characteristics, Decoder-Only models achieve strong performance in generation-centric tasks, including language generation, dialogue response, mathematical and code reasoning, summarization, and question answering. Prominent examples include the GPT series, LLaMA (Touvron et al., 2023a), PaLM (Chowdhery et al., 2023), and Falcon (Almazrouei et al., 2023). In particular, since the advent of GPT3, these models have demonstrated general-purpose capabilities across diverse tasks, such as text generation, code completion, translation, summarization, and question answering.

The Encoder-Decoder architecture consists of an encoder that encodes the input sequence with bidirectional attention, and a decoder that generates outputs autoregressively, referencing the encoded information through cross-attention. This structural design is particularly advantageous for tasks that require comprehensive understanding of the input sequence before generating new outputs, such as machine translation and document summarization. Representative models include T5 (Raffel et al., 2020a), BART (Lewis et al., 2020), and mT5 (Xue et al., 2021). Moreover, recent studies have shown that Encoder-Decoder models trained with multi-task fine-tuning achieve more stable performance than Decoder-Only models in zero-shot transfer scenarios, highlighting their broader potential applicability (Xue et al., 2021).

2.3. Benchmarks and datasets

Evaluating the performance of optimized LLMs requires the utilization of standardized benchmarks and datasets that encompass diverse NLP tasks. Benchmarks verify whether models retain their core capabilities and contribute to enhancing the reproducibility and comparability of research findings. Evaluation is primarily conducted in the following domains:

Machine translation. Evaluates a model's performance in accurately and fluently translating text from one language to another. Standard datasets such as the WMT series (Bojar et al., 2014) are predominantly utilized for this purpose. BLEU, the primary evaluation metric, assesses translation accuracy and fluency by measuring the overlap between model-generated translations and human reference translations. A high BLEU score indicates that the model has produced high-quality translations similar to reference translations, serving as a crucial indicator for maintaining translation quality during multilingual LLM compression. Additional metrics include METEOR (Banerjee & Lavie, 2005), chrF (Popović, 2015), and ROUGE (Lin, 2004).

QA & reading comprehension. Evaluates a model's ability to comprehend given context and generate or identify accurate answers to specific questions based on that understanding. Benchmarks such as SQuAD (Rajpurkar et al., 2016), WikiHop (Welbl et al., 2018),

TriviaQA (Joshi et al., 2017) and OpenBookQA (Banerjee et al., 2019) are utilized. The primary evaluation metric, Exact Match (EM), awards scores only when the model's answer literally matches the ground truth exactly, while the F1 score calculates the harmonic mean of word-level precision and recall between the ground truth and model answers, thereby accounting for partial matches. High EM and F1 scores indicate the model's accurate information extraction and comprehension capabilities.

Reasoning & commonsense. Evaluates a model's logical reasoning, multi-step inference abilities, and commonsense understanding of everyday situations and concepts. These abilities are measured through benchmarks such as BoolQ (Clark et al., 2019), ARC (Clark et al., 2018), HellaSwag (Zellers et al., 2019), PIQA (Bisk et al., 2020), Winogrande (Sakaguchi et al., 2021), SocialIQA (Sap et al., 2019), and CommonsenseQA (Talmor et al., 2018). The primary evaluation metric is accuracy, which measures the proportion of correctly selected answers from given choices for reasoning problems or commonsense questions.

Summarization. Evaluates a model's ability to extract or generate key information from long documents or multiple documents to create concise and semantically coherent summaries. Datasets such as CNN/DailyMail (Hermann et al., 2015), arXiv (Clement et al., 2019), PubMed (Dernoncourt & Lee, 2017), MediaSum (Zhu et al., 2021), and Multi-News (Fabbri et al., 2019) are utilized. The primary evaluation metric, ROUGE, measures the overlap between model-generated summaries and human-written reference summaries.

Code & math & scientific. Evaluates model performance in programming code generation, mathematical problem solving, and scientific question answering. Representative benchmarks include HumanEval (Chen et al., 2021), GSM8K (Cobbe et al., 2021), and Hendrycks' MATH (Hendrycks et al., 2021). Pass@k is utilized for code generation, while accuracy metrics are employed for mathematical and scientific problems.

Long context & retrieval. Evaluates a model's ability to process long documents, retrieve specific information, and capture long-range dependencies. Benchmarks include PG19 (Rae et al., 2019), LongBench (Bai et al., 2023b), Needle-in-a-Haystack (Kamrath, 2023), RULER (Hsieh et al., 2024), MS MARCO (Bajaj et al., 2016), TopiocQA (Adlakha et al., 2022), and QReCC (Anantha et al., 2020). Language modeling-based evaluation is measured using perplexity (PPL), where lower values indicate better long-context comprehension.

Language modeling. Evaluates a model's fundamental language understanding and generation capabilities, specifically how well it predicts the next word. Datasets such as WikiText (Merity et al., 2016a), C4 (Zhu et al., 2023), and Penn Treebank (Marcus et al., 1993) are utilized. The primary evaluation metric, PPL, measures how well the model predicts the next word, where lower PPL values indicate that the model better understands the context of given text and demonstrates greater linguistic fluency. This serves as a fundamental metric for measuring how naturally compressed models generate and understand text.

Benchmark suites. Evaluate models' language understanding, reasoning, and knowledge utilization capabilities by bundling various NLP tasks. Representative benchmarks include GLUE (Wang et al., 2018)'s MRPC (Dolan & Brockett, 2005) and QQP (Sharma et al., 2019), LRA (Tay et al., 2020), and MMLU (Hendrycks et al., 2020). Task-appropriate metrics such as accuracy, F1 score, and Matthews correlation coefficient are applied, with final performance calculated as the average or weighted average of task-specific scores.

3. Efficient module design

Recent research has focused on improving inference speed, memory efficiency, and training stability by optimizing or replacing the core modules described in Section 2.1. As shown in Fig. 2, this section first categorizes efficient module designs and then discusses their impacts on the Transformer framework as well as the resulting performance benefits.

3.1. Normalization

LayerNorm (Ba et al., 2016b) has emerged as the predominant normalization technique among various alternatives (e.g., BatchNorm Ioffe & Szegedy, 2015, RMSNorm Zhang & Sennrich, 2019), primarily due to its robustness to varying sequence lengths. LayerNorm normalizes token-wise features based on their mean and variance, followed by learned scaling and shifting operations, thereby enhancing training stability and accelerating convergence. Initially, LayerNorm was applied after each module; however, as models became deeper, its placement shifted to precede modules, thus facilitating improved gradient flow. More recently, methods have been proposed to dynamically adjust the position of LayerNorm depending on layer depth, highlighting ongoing research efforts aimed at further optimizing training stability for increasingly deep architectures.

LayerNorm has been applied within each sub-layer. It was first adopted in the Post-LN configuration, where normalization is placed after each module. However, this approach was later found to suffer from gradient scale instability as network depth increases. To address this issue, Transformers without Tears (Nguyen & Salazar, 2019) demonstrated that placing LayerNorm before the module (Pre-LN) enables stable training even without warm-up strategies. Furthermore, based on mean-field theory, it was theoretically shown that the Pre-LN (Xiong et al., 2020) configuration maintains stable gradient flow along the residual paths.

Building on these insights, Mix-LN proposes a hybrid architecture that combines the strengths of both approaches: Post-LN in the early layers to amplify gradient signals, and Pre-LN in the later layers to stabilize training. Experimental results show that Mix-LN achieves lower PPL and faster convergence compared to conventional normalization schemes. Nonetheless, despite such structural improvements, LayerNorm still entails computational overhead due to the mean and variance calculations it requires.

In contrast, the computational overhead introduced by the mean and variance calculations, as well as the broadcasting operations inherent to LayerNorm, can pose significant limitations in large-scale models or edge deployment scenarios where computational efficiency is paramount. As a result, recent research has critically re-examined the necessity of LayerNorm, seeking alternative approaches that reduce the associated computational burden without compromising representational capacity.

To address the computational overhead of LayerNorm, recent studies have explored approaches that either remove LayerNorm entirely or replace it with simpler alternatives. AdaNorm (Xu et al., 2019) eliminates the gain and bias parameters and introduces an input-dependent non-linear scaling function $\phi(y_i)$ to stabilize the output distribution without relying on LayerNorm. This formulation is expressed in Eq. (6):

$$\phi(y_i) = C(1 - ky_i), \quad z_i = \phi(y_i) \cdot y_i \quad (6)$$

In this formula, C serves as a parameter that regulates the overall output scale, while k functions as a scale attenuation parameter designed to ensure $\phi(y_i) > 0$ across the range of y_i . AdaNorm demonstrated superior performance compared to LayerNorm in 7 out of 8 benchmark evaluations, achieving notably lower validation loss and enhanced convergence properties, particularly in environments susceptible to overfitting.

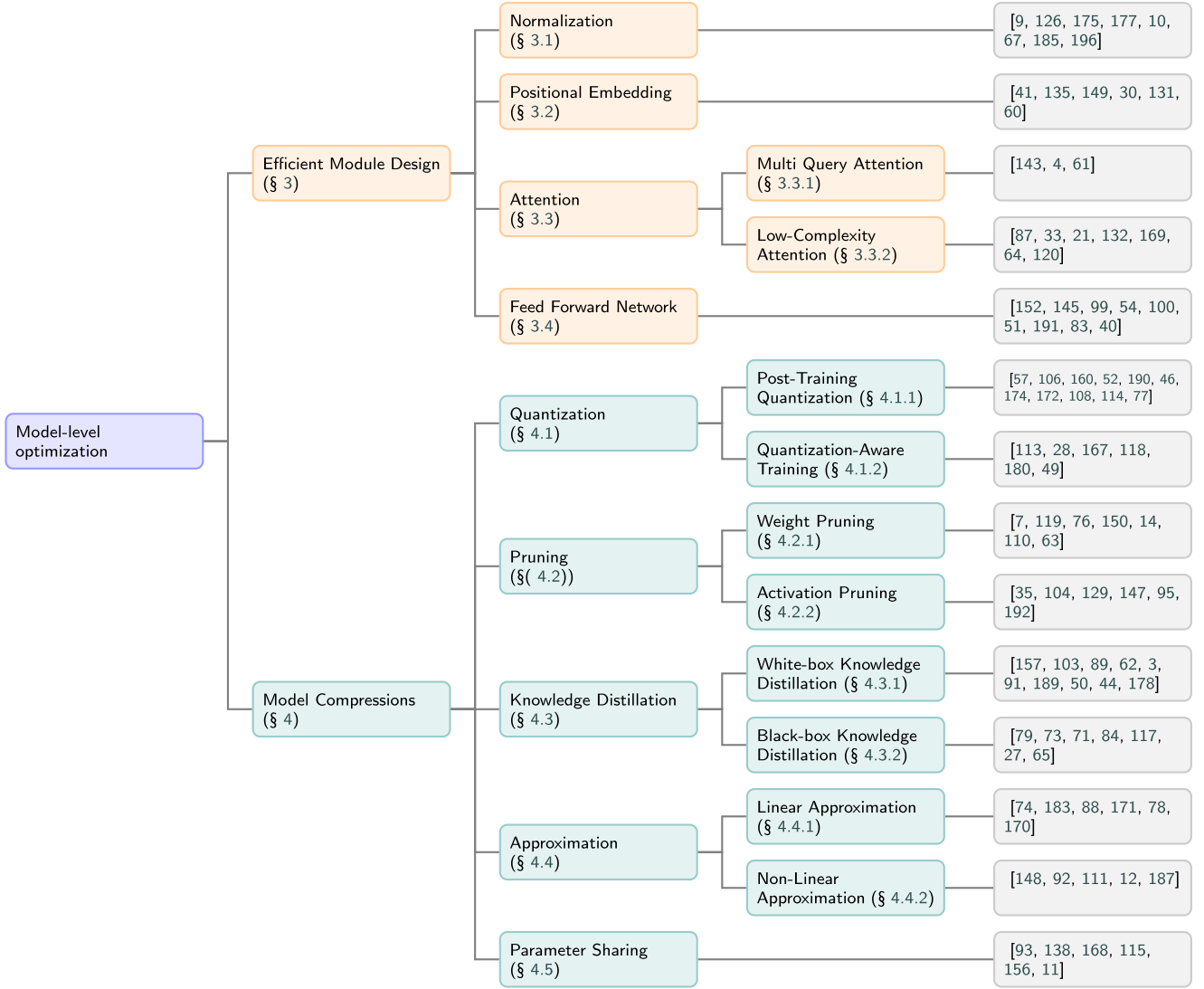


Fig. 2. Taxonomy of model-level optimization techniques.

ReZero (Bachlechner et al., 2021) introduces a learnable scaling parameter α in the residual branch, completely removing the need for LayerNorm while ensuring stability during the early training phase and gradually expanding the model's representational capacity as training progresses. This is defined in Eq. (7).

$$x_{i+1} = x_i + \alpha_i \cdot \text{sublayer}(x_i) \quad (7)$$

Subsequently, nGPT (Loshchilov et al., 2024) demonstrates that by normalizing all vectors—including embeddings, attention outputs, and MLP outputs—to unit norm, it is possible to maintain stable output scaling and enable fast, stable training without relying on LayerNorm or weight decay.

Furthermore, Remove GPT2's LayerNorm (Heimersheim, 2024) reported that, after removing LayerNorm and applying fine-tuning, GPT2's accuracy on the HellaSwag benchmark decreased only marginally from 30.0% to 29.5%. These findings suggest that while LayerNorm contributes critically to stabilizing the initial training process, it may be unnecessary—or even detrimental to efficiency and expressiveness—during later stages of training in deep networks.

Meanwhile, RMSNorm (Zhang & Sennrich, 2019) has been introduced as an approach to reduce computation while preserving stability. It performs normalization using only the root mean square of the input, omitting the mean calculation and thereby significantly lowering com-

putational complexity compared to LayerNorm. This simplification not only reduces latency but also improves memory access efficiency, establishing RMSNorm as the de facto standard adopted by many state-of-the-art large language models, including the LLaMA family, Mistral (Jiang et al., 2023a), and DeepSeek (Liu et al., 2024a).

In addition, Dynamic Tanh (Zhu et al., 2025) builds on the observation that the input-output relationship of LayerNorm closely resembles the behavior of the tanh function. It applies a learnable scaling coefficient to the input, followed by a tanh activation, enabling stable training without the need for complex mean and variance calculations. Empirical studies show that Dynamic Tanh achieves performance on par with—or even superior to—LayerNorm across diverse models and tasks.

3.2. Positional embedding

PE can be broadly classified into two primary approaches based on how positional information is represented and utilized. The first approach is absolute positional embedding (APE), which encodes and utilizes the absolute position index of each token within the sequence. The second is relative positional embedding (RPE), which models positional information by focusing on the relative distances or relationships between token pairs rather than the absolute position of individual tokens. While vanilla transformer employs APE to inject token positions, several

inherent limitations exist. Since PE is combined with input embeddings at the initial stage and is only generated up to a predefined maximum sequence length, this imposes constraints on processing longer sequences. Furthermore, considering only the absolute position of each token, it fails to explicitly model relative distances or relationships between tokens. RPE emerged to address these issues, enabling models to learn richer information about positional relationships compared to APE and improving extrapolation performance to generalize to longer sequences without constraints on context length.

The pioneering work on RPE by Shaw et al. (2018) proposed a method that directly incorporates relative position information between tokens by modifying the attention mechanism itself. The core of this approach is to directly inject relative position information into the computation of keys (K) and values (V) when calculating attention scores, as shown in Eq. (8). Experimental results demonstrate that the proposed RPE technique yields improvements of 1.3 BLEU over vanilla transformer using APE on WMT 2014 English-to-German.

$$\begin{aligned} f_Q(x_m) &:= \mathbf{W}_Q x_m \\ f_K(x_n, n) &:= \mathbf{W}_K(x_n + \tilde{\mathbf{p}}_r^K) \\ f_V(x_n, n) &:= \mathbf{W}_V(x_n + \tilde{\mathbf{p}}_r^V) \end{aligned} \quad (8)$$

Transformer-XL (Dai et al., 2019) introduced the Segment-Level Recurrence mechanism to overcome the fixed context length limitation of APE. In doing so, the authors identified a temporal confusion problem: tokens at identical indices across different segments shared the same positional encodings, making them indistinguishable under conventional APE. To resolve this issue, they proposed a novel RPE approach. This method decomposes the vanilla transformer's attention score computation into four terms as shown in Eq. (9): (a) content-to-content, (b) content-to-position, (c) position-to-content, and (d) position-to-position interactions, and subsequently modifies these terms to incorporate relative position information.

$$\mathbf{Q}_m^T \mathbf{K}_n = \underbrace{\mathbf{x}_m^T \mathbf{W}_Q^T \mathbf{W}_K \mathbf{x}_n}_{(a)} + \underbrace{\mathbf{x}_m^T \mathbf{W}_Q^T \mathbf{W}_K \mathbf{p}_n}_{(b)} + \underbrace{\mathbf{p}_m^T \mathbf{W}_Q^T \mathbf{W}_K \mathbf{x}_n}_{(c)} + \underbrace{\mathbf{p}_m^T \mathbf{W}_Q^T \mathbf{W}_K \mathbf{p}_n}_{(d)} \quad (9)$$

Terms (b) and (d) are replaced with fixed sinusoidal relative position encodings, and learnable parameter vectors u and v are introduced, which are uniformly applied across all query positions. Experimental results demonstrate consistent performance improvements over RNN, LSTM-based models, and vanilla transformer across various model sizes on enwik8 (Mahoney, 2011), text8 (Mahoney, 2011), WikiText-103, One Billion Word (Chelba et al., 2013), and The Penn Treebank datasets. Nevertheless, while Transformer-XL alleviates the fixed context limitation, its segment-level recurrence restricts gradient flow across segments and remains bounded by GPU memory. Moreover, the quadratic complexity of attention still poses efficiency challenges.

T5 (Raffel et al., 2020a) directly adds a learnable scalar bias value corresponding to the relative offset between K and Q to the attention score, while removing all terms except term (a). The attention scores are expressed as shown in Eq. (10).

$$\mathbf{Q}_m^T \mathbf{K}_n = \mathbf{x}_m^T \mathbf{W}_Q^T \mathbf{W}_K \mathbf{x}_n + b_{i,j} \quad (10)$$

Nevertheless, while this approach simplifies implementation, its bucketed relative bias design limits the ability to capture long-range dependencies, and the framework remains highly resource-intensive.

The RPE approaches proposed in Transformer-XL and T5 demonstrated the potential for more flexible and effective positional information modeling by focusing on relative distances between tokens rather than absolute position values themselves. In particular, T5's simple scalar bias approach offers advantages in terms of computational efficiency and implementation simplicity, providing fundamental ideas that have been crucial for improving and advancing positional encoding methods in subsequent research, such as DeBERTa.

Rotary Position Embedding (RoPE) (Su et al., 2024) overcomes the limitations of additive position encoding methods by encoding absolute

positions through multiplicative rotation matrices. This approach naturally captures relative position dependencies within self-attention. The core idea of RoPE is to divide the d -dimensional embedding vector into $d/2$ two-dimensional subspaces and apply rotational transformations using rotation angles $i \cdot m \cdot \theta$ corresponding to the absolute position m of tokens in each subspace.

$$\begin{aligned} f_Q(x_m, m) &= (\mathbf{W}_Q x_m) e^{im\theta} \\ f_K(x_n, n) &= (\mathbf{W}_K x_n) e^{in\theta} \\ g(x_m, x_n, m - n) &= \text{Re}[(\mathbf{W}_Q x_m)(\mathbf{W}_K x_n)^* e^{i(m-n)\theta}] \end{aligned} \quad (11)$$

Applying RoPE to the Q and K as shown in Eq. (11) yields rotation matrices for each. Computing their inner product results in a form that depends solely on the relative position $n - m$. RoPE exhibits a long-term decay property where the upper bound of inner product values decreases as relative distance increases, and offers the advantage of direct compatibility with linear attention. Experimental results demonstrate its effectiveness with a 0.2 BLEU improvement over vanilla transformer on WMT14 En-De, and F1 score improvements of 0.6 and 15.2 on MRPC and QQP, respectively, compared to BERT-base.

Chen et al. (2023) proposed Position Interpolation (PI) to address the dramatic performance degradation that occurs when directly inputting sequences exceeding the training-time maximum context length. This technique linearly interpolates position indices to fit within the original training range, as expressed in Eq. (12).

$$\mathbf{f}'(\mathbf{x}, m) = \mathbf{f}\left(\mathbf{x}, \frac{mL}{L'}\right). \quad (12)$$

When processing longer contexts, instead of directly using the actual token position m in the RoPE function, PI uses the linearly scaled position $m * L/L'$ as input. In passkey retrieval tasks, models achieved near-perfect passkey retrieval up to the target context length of 32k with only PI and 200 fine-tuning steps, demonstrating effective long-range information utilization capabilities.

Yet another RoPE extension (YaRN) (Peng et al., 2024) points out the problems of high-frequency information loss and reduced discrimination between nearby tokens in the PI approach, and improves existing RoPE interpolation methods by proposing YaRN, an efficient context window extension technique that incorporates pre-softmax attention scaling.

$$r(d) = \frac{L}{\lambda_d} = \frac{L}{2\pi b^{\frac{2d}{|D|}}} \quad (13)$$

As shown in Eq. (13), YaRN differentially interpolates each dimension of RoPE based on the parameter λ and the ratio L'/L between the target context length L' and original length L , rather than uniformly scaling all dimensions. Experimental results show that the YaRN-fine-tuned LLaMA2-13B model maintains a PPL of 2.24 up to 128k length, outperforming existing methods with minimal degradation on general benchmarks. However, the method requires careful heuristic hyperparameter tuning, which may limit reproducibility and general applicability across different models.

Contextual Position Encoding (CoPE) (Golovneva et al., 2024) addresses the limitation of existing PE methods that measure positions based solely on token counts, proposing a novel position encoding approach that is dynamically computed according to context. CoPE calculates gate values for each query and key based on contextual information, and dynamically determines the contextual position p_{ij} of keys relative to queries through the cumulative sum of these values. The embedding for positions represented as real numbers is obtained through interpolation using predefined integer position embeddings z , which is then incorporated into attention score computation as follows:

$$a_{ij} = \text{Softmax}(\mathbf{Q}_i^T \mathbf{K}_j + z_i[p_{ij}]). \quad (14)$$

Experimental results demonstrate PPL improvements over baselines on WikiText-103 and code language modeling, with particularly strong performance in extrapolation capabilities. However, since the study evaluates only small-scale models, further validation is required for large models and diverse downstream tasks.

3.3. Attention

Attention mechanisms exhibit quadratic computational complexity with respect to sequence length, causing computational bottlenecks when processing long contexts. Research on attention module modifications to address this issue can be broadly categorized into two main directions: multi-query attention and low-complexity attention.

3.3.1. Multi query attention

Multi-Query Attention (MQA) (Shazeer, 1911) addresses the memory bandwidth bottleneck of MHA during incremental decoding, where each head redundantly stores its own K and V matrices, causing excessive cache usage and slow inference (Vaswani et al., 2017). To overcome this limitation, MQA shares common K and V matrices across all heads while keeping only Q separate. This design alleviates the memory bandwidth bottleneck by reducing redundant key-value storage, thereby maximizing efficiency. Experimental results show that MQA improves inference speed by up to 12×, though it comes with minor performance degradation, with PPL increasing by about 0.3 compared to MHA.

Grouped-Query Attention (GQA) (Ainslie et al., 2023) groups multiple heads such that each group shares the same K and V matrices, thereby reducing memory and computational costs compared to MHA, while retaining more expressive capability than MQA. By adjusting the number of groups, GQA allows a flexible trade-off between efficiency and performance. Experimental results show that GQA, when trained in a grouped-query configuration using only 5% of the pretraining compute of the original T5-XXL model, achieves inference speeds comparable to MQA while maintaining similar accuracy to T5-XXL (Raffel et al., 2020b) on tasks such as CNN/DailyMail, arXiv, PubMed, MediaSum, Multi-News, WMT, and TriviaQA.

Multi-Token Attention (MTA) (Golovneva et al., 2025) is a structure designed to overcome the limitations of traditional self-attention by applying two lightweight convolutions to the attention score matrix. A 3×3 2D convolution is used to capture contextual information between adjacent tokens, while a 1D convolution integrates complementary information across heads. The overall parameter increase is minimal, at less than 0.01%. Experimental results show that MTA achieves up to a 15% improvement in accuracy compared to the vanilla transformer and up to a 0.6 reduction in PPL when extending sequence length.

3.3.2. Low-Complexity attention

Low-complexity attention focuses on redesigning the attention computation itself to reduce the quadratic complexity inherent in traditional attention mechanisms. Within this trend, various low-complexity attention techniques have emerged, aiming to lower the computational cost to linear or sub-quadratic levels while preserving model performance. These methods can be broadly categorized into two main types: kernel-based attention and low-rank attention.

Kernel-based attention. Kernel-based attention replaces the softmax operation used in traditional self-attention with a kernel function $\phi(\cdot)$ and restructures the computation based on inner products. Specifically, the attention output is computed by first accumulating $\phi(K_j)^T V_j$ over all j , and then multiplying the result with $\phi(Q_i)$. This approach leverages the associativity of matrix multiplication to reorder operations, enabling linear computational complexity $\mathcal{O}(N)$ with respect to the input length N .

Linear Transformer (Katharopoulos et al., 2020) proposes an efficient self-attention mechanism based on kernelized inner-product attention, reducing the computational complexity from the original softmax-based $\mathcal{O}(N^2)$ to $\mathcal{O}(N)$. The attention output in this architecture is defined as:

$$V'_i = \frac{\phi(Q_i)^T \sum_{j=1}^N \phi(K_j) V_j^T}{\phi(Q_i)^T \sum_{j=1}^N \phi(K_j)}, \quad (15)$$

where Eq. (15) approximates the probabilistic normalization of softmax attention weights. By separately computing the numerator and denominator, this formulation effectively reduces computational time and memory consumption. Moreover, when applying a sequential attention structure with causal masking, Linear Transformer achieves up to 4000× faster autoregressive predictions on the enwik8 benchmark compared to the vanilla Transformer, while maintaining comparable performance. Experimental results on WikiText-103 further demonstrate that it preserves performance similar to the original model, while accelerating training and inference speeds by up to 2×.

Performer (Choromanski et al., 2020) reinterprets softmax attention as a Gaussian kernel ϕ and approximates the exponential inner product using Fast Attention Via positive Orthogonal Random features (FAVOR+), reducing both stages of the computation to $\mathcal{O}(n)$. It first computes the KV product to reduce the intermediate dimensionality, and then multiplies it with the query Q to obtain the final output. On WikiText-103, Performer achieves a PPL of 26.8 while improving inference speed by approximately 2×.

TFLA (Beck et al., 2025) divides the input sequence into fixed-length chunks and computes memory states step-by-step using inter-chunk parallelism, which allows each chunk to be processed in parallel. Then, within each chunk, it applies intra-chunk parallelism by splitting it into smaller tiles and parallelizing three matrix multiplications, enabling efficient computation and memory usage even with much larger chunk sizes than previous methods. The attention computation in this architecture is expressed as:

$$H^{(k)} = \underbrace{Q^{(k)} K^{(k)T} V^{(k)}}_{\text{Intra-chunk parallel (attention)}} + \underbrace{Q^{(k)} C_{k-1}}_{\text{Inter-chunk recurrent (memory state)}} \quad (16)$$

Here, $Q^{(k)}$, $K^{(k)}$, and $V^{(k)}$ refer to the query, key, and value matrices of the k -th chunk, respectively, and C_{k-1} denotes the memory state computed from the previous chunk. This design significantly reduces GPU memory I/O bottlenecks and enhances the compute intensity, achieving faster training and inference speeds than FlashAttention (Dao, 2024). On inference tasks using GPT-style models trained on The Pile (Gao et al., 2020), TFLA demonstrates up to 2.1× faster inference and over 30% improvement in kernel performance compared to FlashAttention on benchmarks such as WikiText-103 and PG19.

Previous attention methods approximate the softmax inner product or introduce sparsity, but often degrade on medium-length sequences (Xiong et al., 2021). RFA (Peng et al., 2021), a linear-time mechanism, addresses this by approximating $\exp(q \cdot k)$ with random Fourier features, (i.e., $\phi(q) \cdot \phi(k)$). The computation is restructured by first evaluating KV to reduce the dimensionality and then multiplying with Q , thereby reducing the overall complexity to $\mathcal{O}(n)$. In addition, RFA introduces a gated variant that encourages the model to attend more strongly to recent tokens such as the immediately preceding input, mitigating the expressiveness degradation caused by the approximation. Experimental results show that RFA achieves comparable or better accuracy than vanilla transformers on benchmarks such as WikiText-103 language modeling, WMT machine translation, and the Long Range Arena. In particular, it attains up to 2× faster decoding speed over baselines in machine translation, and for long sequences of length 2048, it delivers 12× faster decoding with over 90% memory savings, demonstrating outstanding efficiency in long-context processing.

Low-rank attention. Low-rank attention reduces computation and memory usage by applying a linear projection to compress the token dimension (n) of the K V matrices into a smaller dimension (k). This reduces the computational complexity of attention from the original $\mathcal{O}(n^2 d)$ to $\mathcal{O}(nkd)$, enabling efficient computation even with long input sequences.

Linformer (Wang et al., 2020) proposes a structure that reduces the self-attention computation of Transformers from $\mathcal{O}(n^2)$ to $\mathcal{O}(n)$ by applying learnable linear projections to the K and V matrices. In each attention head, K and V are projected into a lower-dimensional space using

projection matrices $E_i, F_i \in \mathbb{R}^{k \times n}$, and the full attention computation is defined as follows:

$$\text{Attention}(QW_i^Q, E_i K W_i^K, F_i V W_i^V) = \text{softmax}\left(\frac{QW_i^Q(E_i K W_i^K)^T}{\sqrt{dk}}\right) \cdot F_i V W_i^V \quad (17)$$

Experimental results show that Linformer achieves comparable or even better accuracy than RoBERTa on GLUE (Wang et al., 2018) tasks, while improving inference speed and memory usage by up to 20× on long sequences.

The self-attention mechanism in Transformers computes interactions between all token pairs, leading to excessive computation and memory usage as sequence length increases, which creates a bottleneck for long-context processing (Beltagy et al., 2020). FLuRKA (Gupta et al., 2024) addresses this limitation by proposing a novel attention architecture that integrates low-rank and kernel approaches in an optimal sequence. The key idea is based on the observation that the attention matrix obtained from kernel approximation is inherently low-rank. FLuRKA first applies low-rank compression and then performs kernel approximation, thereby exploiting the computational efficiency of both techniques simultaneously. This design reduces the costs of both linear transformations and softmax approximations, enabling faster speed compared to existing methods. Experimental results show that FLuRKA achieves up to 3.3× and 1.7× speedups over pure low-rank and kernel methods, respectively, while surpassing full attention by up to 25× and flash attention by up to 23×. On benchmarks such as WikiText-103 language modeling, GLUE, Long Range Arena, machine translation, and ImageNet classification, FLuRKA matches or outperforms prior approaches, and on some tasks, it even surpasses both low-rank and kernel baselines. These results demonstrate that FLuRKA is a high-quality linear attention mechanism that achieves both efficiency and expressiveness.

Luna (Ma et al., 2021) is a linearized attention mechanism designed to overcome the inefficiency of softmax-based attention, whose time and memory complexity grow quadratically with the sequence length and becomes impractical for long-context processing (Tay et al., 2022). Previous efficiency-oriented approaches have attempted to mitigate this issue, but they typically suffer from significant accuracy degradation. To address these limitations, Luna introduces a dual attention structure that first packs the input sequence into a fixed-length projection vector and then unpacks this compressed representation with respect to the original queries. In the pack stage, the input context is summarized through a fixed-length query P , while in the unpack stage, the original query X is used to restore the final output, effectively reformulating the attention operation into a linear compress-restore process. This design reduces the computational complexity to $O(n)$ while maintaining both accuracy and efficiency. Experimental results demonstrate that Luna achieves up to 5.5× faster speed and 90% lower memory usage compared to the vanilla Transformer on benchmarks such as LRA, WMT'14, and GLUE, while preserving equal or superior accuracy.

3.4. Feed forward network

FFNs have been extensively studied to enhance their capacity to capture complex dependencies in input representations, while maintaining computational efficiency. A widely used approach for enhancing this capacity is the gated linear unit (GLU), which splits the output of the first linear transformation into two parallel paths: one passed through directly, and the other processed by an additional linear layer and a non-linearity to generate a gating signal. The two paths are then combined via element-wise multiplication, allowing the gating mechanism to selectively suppress irrelevant activations and emphasize salient features. By appropriately adjusting the intermediate hidden size, GLU variants can often achieve superior performance compared to standard FFNs at an equivalent computational cost (Shazeer, 2020). Furthermore, several GLU extensions—such as ReGLU, GEGLU, and SwiGLU—have introduced alternative activation functions, further improving accuracy.

MobileBERT (Sun et al., 2020), a compact variant of BERT (Devlin et al., 2019), introduces a stacked FFN structure to enhance modeling capacity without increasing hidden size, enabling efficient operation on mobile devices. Instead of increasing the hidden size, MobileBERT stacks multiple FFNs sequentially within a single Transformer block, thereby achieving near-original performance in mobile environments.

Meanwhile, a growing alternative is to replace the FFN with a mixture-of-experts (MoE) structure, which enables token-adaptive sparse computation (Shazeer et al., 2017). MoE is a structure that selectively utilizes multiple FFNs, referred to as experts. A newly introduced gating network employs Top- k routing to choose the top k experts to activate for each token. While all experts are stored in memory, only a few are activated per token, maintaining a computational cost similar to dense FFNs. The input hidden state h is fed into a linear gating function, producing selection probabilities via a Softmax. The outputs from the selected experts are aggregated with weighted sums to produce the final output o , which is then passed to the next layer via residual connection.

GShard (Lepikhin et al., 2021) was the first to incorporate MoE into Transformer architectures by replacing half of the FFNs with MoE layers and activating $k = 2$ experts per token. This design increases model capacity while preserving sparse per-token computation. However, the softmax-based gating mechanism often results in load imbalance, where certain experts become overloaded. GShard addresses this by limiting the number of tokens each expert can process and passing through the original hidden state for any overflow.

Switch Transformer (Fedus et al., 2022) simplifies this by using Top-1 routing, allowing each token to activate only a single expert. This significantly reduces both computation and communication overhead. During training, a load balancing loss is added to evenly distribute the workload across experts and maintain stable inference latency.

BASE Layers (Lewin et al., 2021) aim to resolve the load imbalance issue fundamentally by enforcing equal token-to-expert allocation during training. Each expert is assigned a unique embedding, and tokens are routed based on similarity such that all experts process an equal number of tokens. This reduces communication cost and improves both load distribution and model convergence.

GLaM (Du et al., 2022) targets the high training cost and unstable load balancing issues of early MoE models. It replaces the FFN in each Transformer block with a 64-expert Top-2 MoE and distributes large experts via 2D sharding so that only the active parameters per token are used. As a result, it reduces inference FLOPs to about half of GPT3 and significantly lowers training energy, while outperforming GPT3 in average performance across 29 benchmarks. However, its applicability is limited to large-scale models, with practical benefits emerging only from around 8B parameters and above.

MoEfication (Zhang et al., 2022b) introduces a static MoE mechanism by clustering the FFN's intermediate weights into M equally sized blocks based on neuron co-activation patterns, and treating these blocks as experts. During inference, either the similarity between the current hidden state and the average weight of each expert or a lightweight MLP router is used to select the top- k experts. This reduces both FLOPs and memory access by a factor of k/M .

Mixtral (Jiang et al., 2024) addresses the training complexity of MoE layers, expert load imbalance, and the risk of generalization degradation when scaling to many experts. It expands each FFN into eight experts and adopts a sparse MoE with Top-2 gating, activating only two experts per token and combining their outputs. This preserves a large total parameter budget while constraining per-token active parameters for efficiency. Reported results show that Mixtral-8×7B matches or surpasses LLaMA2-70B on mathematics, code generation, and multilingual benchmarks, and it supports context lengths up to 32k tokens.

DeepSeekMoE (Dai et al., 2024) addresses expert redundancy by dividing the FFN's hidden dimension into k parts, each forming a $1/k$ -sized expert. All tokens pass through a shared expert, followed by selectively routed experts. This separation enables the shared expert to capture

general knowledge (e.g., grammar), while routed experts specialize in task-specific information, achieving both efficiency and expressiveness.

4. Model compressions

Model compression aims to improve inference speed and reduce deployment costs by minimizing performance loss from the original model while reducing model size and FLOPs. Fig. 2 presents a taxonomy of the core compression techniques discussed in this section, and Fig. 3 provides a visual illustration of quantization, pruning, KD, approximation, and parameter sharing according to this taxonomy.

4.1. Quantization

Quantization is a widely used compression technique that reduces the precision of model parameters and activations from floating-point to low-bit fixed-point or integer formats. This reduction in bit-width lowers both memory usage and computational cost, making quantization especially effective for compressing LLMs, which demand substantial resources. Depending on when it is applied, quantization methods are typically classified into post-training quantization (PTQ) and quantization-aware training (QAT). PTQ quantizes a pretrained model without further training, while QAT integrates quantization operations into the training process to jointly optimize quantization parameters and preserve accuracy. PTQ methods can be further categorized into weight-only and weight-activation quantization. Table 1 summarizes and compares recent quantization methods on LLaMA2-7B by reporting WikiText-2 PPL and averaged zero-shot accuracy over five common-sense reasoning benchmarks (i.e., PIQA, ARC-e, ARC-c, WinoGrande, and HellaSwag).

4.1.1. PTQ

PTQ compresses pretrained models by reducing their precision to low-bit formats without requiring retraining, thereby substantially lowering both model size and inference cost. Due to its minimal computational and time overhead compared to QAT, PTQ has become the most practical and widely adopted quantization strategy for large-scale LLMs with billions of parameters. Recent PTQ methods aim to preserve accuracy by estimating quantization parameters offline using a small calibration dataset. PTQ is generally classified into weight-only and weight-activation quantization. Weight-only quantization compresses only the model weights while keeping activations in higher precision, effectively

mitigating memory bottlenecks. In contrast, weight-activation quantization applies quantization to both weights and activations, maximizing overall efficiency in both computation and memory usage.

Weight-only quantization. While simple rounding methods such as rounding-to-nearest methods (RTN) suffer from severe accuracy degradation in large-scale models, GPTQ (Frantar et al., 2022) refines important weights more precisely by leveraging Hessian-based sensitivity estimation. Designed with a strong emphasis on memory efficiency and one-shot applicability, GPTQ represents the first successful quantization technique applied to a 175B-parameter GPT model. Specifically, GPTQ builds on the OBQ (Frantar & Alistarh, 2022) framework and performs quantization in the same fixed column-wise order across all rows, which enables the reuse of a shared Hessian inverse matrix and thereby reduces computational cost. Furthermore, through module-level batch processing and the use of Cholesky decomposition, GPTQ minimizes both memory and compute requirements while still maintaining quantization stability. As a result, GPTQ preserves much higher accuracy compared to simple RTN and empirically demonstrates that sub-4-bit quantization is practically achievable even for models with over 175 billion parameters.

In contrast to sensitivity-based correction of GPTQ, AWQ (Lin et al., 2024) proposes an activation-aware scaling approach to preserve critical weights. It identifies weights in channels with large activation values as salient weights and applies per-channel scaling to them. Specifically, salient weights are amplified before quantization, while their corresponding activations are inversely scaled, ensuring that the computational results remain unchanged while effectively reducing quantization error. Using this method, AWQ achieves a PPL of 3.95 on the WikiText-2 dataset with a 3-bit quantized LLaMA-65B model. Moreover, when combined with GPTQ (Frantar et al., 2022), AWQ enables extreme 2-bit quantization of the OPT-30B model, reaching a PPL of 11.38 on WikiText-2. These results suggest that GPTQ's reconstruction-based correction and AWQ's salient-weight preservation are complementary techniques.

Although GPTQ and AWQ demonstrate stable performance primarily in the 3-4-bit regime, they still face significant accuracy degradation under more extreme 2-bit quantization. To address this limitation, QuIP# (Tseng et al., 2024) focuses on the outlier problem in weight distributions. By applying a Randomized Hadamard Transform, QuIP# reshapes weight distributions into a Gaussian-like form without outliers.

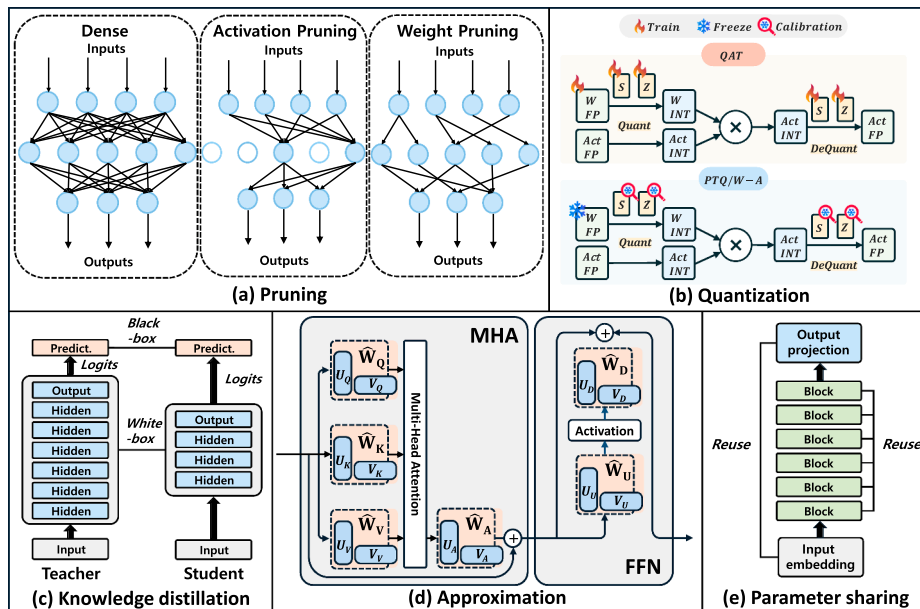


Fig. 3. Taxonomy of model-level optimization and compression techniques for LLMs.

Table 1
Comparison of quantization methods for LLaMA2-7B: perplexity and 0-shot accuracy.

Method	PTQ/QAT	Target	W #-Bit	A #-Bit	PPL	0-shot
Baseline	–	–	16	16	5.47	66.21
Frantar et al. (2022)	PTQ	Weight-Only	4	16	5.83	66.56
Lin et al. (2024)	PTQ	Weight-Only	4	16	6.15	67.47
Tseng et al. (2024)	PTQ	Weight-Only	3	16	5.79	62.85
Egiazarian et al. (2024)	PTQ	Weight-Only	2	16	6.59	57.28
Zhang and Shrivastava (2025)	PTQ	Weight-Only	2.01	16	15.51	50.8
Dettmers et al. (2022)	PTQ	Weight-Activation	8	8	–	–
Xiao et al. (2023)	PTQ	Weight-Activation	6	6	6.2	60.99
Wei et al. (2023)	PTQ	Weight-Activation	6	6	5.9*	–
Liu et al. (2023)	PTQ	Weight-Activation	4	4	11.75	51.60
Liu et al. (2025b)	PTQ	Weight-Activation	4	4	5.9	67.34
Hu et al. (2025b)	PTQ	Weight-Activation	4	4	5.6	67.27
Liu et al. (2024c)	QAT	Weight-Activation	4	8	10.9*	67.18*
Chen et al. (2025)	QAT	Weight-Only	2	16	7.19	59.5
Wang et al. (2023)	QAT	Weight-Activation	1	8	–	–
Ma et al. (2024)	QAT	Weight-Activation	1.58	8	–	–
Xu et al. (2024b)	QAT	Weight-Activation	1	8	9.73	50.06
Dong et al. (2025)	QAT	Weight-Activation	0.55	16	27.93	–

PPL: evaluated on the WikiText-2 language modeling benchmark.

0-shot: averaged accuracy over five common-sense reasoning benchmarks (PIQA, ARC-e, ARC-c, WinoGrande, and HellaSwag).

*: results obtained on LLaMA-7B.

It further introduces a lattice-based codebook that groups weights into 8-dimensional vectors for vector-wise quantization, achieving lower quantization error compared to conventional scalar quantization. With a size under 1 KB, the codebook is well-suited for GPU L1 cache, enabling efficient and fast inference. In addition, QuIP# performs sequential fine-tuning within Transformer blocks at the layer level rather than across the entire model, thereby reducing computational cost while allowing a 3-bit quantized model to outperform 4-bit models in terms of PPL.

Extending beyond this, AQLM (Egiazarian et al., 2024) introduces a learnable additive quantization framework that leverages input-output calibration data to preserve layer outputs. Each weight group is approximated as the sum of vectors selected from multiple codebooks, with both codes and codebooks jointly optimized. Unlike prior additive quantization methods, which were limited to layer-level optimization, AQLM jointly optimizes codebooks at the Transformer block level to reduce intra-layer error. As a result, AQLM achieves a PPL of 3.36 on the WikiText-2 dataset with LLaMA2-70B under an average 3.01-bit quantization, outperforming GPTQ (Frantar et al., 2022). Moreover, even under an average 2.07-bit quantization, AQLM maintains practical accuracy with a PPL of 3.94, highlighting its robustness in extremely low-bit scenarios.

LeanQuant (Zhang & Shrivastava, 2025) observes that prior methods may incur substantial loss increases when outliers appear in the inverse Hessian diagonals, as such outliers disproportionately amplify the quantization error of specific weights. To mitigate this issue, LeanQuant introduces a loss-error-aware grid that reallocates quantization grid points in proportion to each weight's importance. For non-uniform quantization, it learns a weighted k-means-based loss-aware non-uniform grid, whereas for affine quantization, it employs a search-based loss-aware affine grid. This strategy preserves the precision of critical weight regions that are difficult to capture using conventional min-max grids, yielding an impressive PPL of 3.42 on WikiText-2 under a 2-bit quantization setting. Moreover, LeanQuant demonstrates strong compatibility and practicality by enabling the Llama3.1-405B model to run within 65.4 GB of GPU memory.

Overall, recent weight-only quantization research in LLM PTQ has primarily centered on addressing outlier-induced degradation, expanding from GPTQ's Hessian-based sensitivity estimation toward activation-aware scaling (e.g., AWQ Lin et al., 2024) and learnable, codebook-based additive structures (e.g., AQLM Egiazarian et al., 2024). Within this progression, LeanQuant (Zhang & Shrivastava, 2025) provides a new direction by retaining the GPTQ framework while explicitly learning the

quantization grid to minimize loss-error. Its competitive performance in the 2-bit regime further underscores the growing feasibility of ultra-low-precision quantization for practical large-scale LLM deployment.

Weight-activation quantization. Early approaches to weight-activation quantization primarily focused on enhancing precision by refining the granularity of the quantization units. More recent research, however, has shifted attention toward the recognition that outlier distributions in LLMs represent a key contributor to accuracy degradation. Notably, analyses have revealed that activation outliers stem less from the magnitude of individual values and more from range imbalances across channels. This insight has driven the development of advanced correction strategies that extend beyond simple scaling, incorporating techniques such as shifting, distribution rebalancing, and even architectural adjustments to the channel structure itself.

GPT3.int8() (Dettmers et al., 2022) introduces vector-wise quantization to resolve the severe performance degradation observed when applying conventional 8-bit quantization to models larger than 2.7B parameters. This method conceptualizes matrix multiplication as a set of independent inner products, applies individual scaling factors to each row of the input matrix and each column of the weight matrix, and then reconstructs the results through their outer products. In addition, for models larger than 6.7B parameters, GPT3.int8() adopts a mixed-precision scheme in which approximately 0.1% of the outliers in the hidden dimension are computed in 16-bit precision, while the remaining values are processed in 8-bit, thereby enabling quantization without accuracy loss even for models up to 175B parameters. Through this approach, GPT3.int8() makes 8-bit quantization feasible for GPT3-175B and establishes the starting point for weight-activation quantization in large-scale LLMs.

On the other hand, SmoothQuant (Xiao et al., 2023) is based on the observation that activation outliers excessively expand the quantization range and thereby reduce precision. To address this, SmoothQuant proposes a per-channel smoothing technique that mathematically shifts activation outliers onto the weights through equivalent transformations. The required scaling factors are precomputed using calibration data, allowing the method to be applied with no runtime overhead. As a result, SmoothQuant enables W8A8 quantization of the OPT-175B model while maintaining accuracy close to full precision, establishing itself as a representative anchor solution for the activation outlier problem.

Subsequent research has introduced more sophisticated distribution correction strategies to overcome the limitations of these anchor

methods. Outlier Suppression + (OS+) (Wei et al., 2023) addresses the asymmetric outlier problem that SmoothQuant fails to handle by combining channel-wise shifting and scaling. The transformations are absorbed into the weights and biases of the subsequent layer, allowing the method to be applied without additional runtime overhead. As a result, OS+ maintains full-precision accuracy even under W6A6 quantization and achieves up to $1.5\times$ inference speedup on certain models.

QLLM (Liu et al., 2023) further mitigates outliers by flattening distributions through Adaptive Channel Reassembly, which splits channels with concentrated outliers into multiple sub-channels and merges similar channels. In addition, it introduces efficient gradient-based error correction, which learns quantization errors at the layer level using gradients and subsequently merges the correction matrices into the weights, thereby eliminating inference overhead. With this design, QLLM achieves both faster training and higher accuracy compared to prior methods, demonstrating strong performance under W4A4 and W2A16 quantization settings.

Despite various techniques proposed to effectively suppress outliers, a fundamental limitation remains: outliers are still distributed non-uniformly across the data. To address this issue, recent studies have moved beyond simple scaling and shifting, introducing learned transformations that directly optimize the distributional structure over the entire space.

SpinQuant (Liu et al., 2025b) optimizes learnable orthonormal rotation matrices on the Stiefel manifold to reshape activation and weight distributions into a more Gaussian-like and quantization-friendly form. This global smoothing of outliers across residual and attention blocks enables the model to achieve accuracy close to full precision even under the W4A4KV4 configuration.

Building on this direction, OSTQuant (Hu et al., 2025b) points out that previous methods fail to quantitatively account for how transformed distributions utilize the entire quantization space. To remedy this, it introduces a new metric—the quantization space utilization rate (QSUR)—which measures the proportion of the quantization range effectively occupied by the data. Based on this metric, OSTQuant learns an equivalent transformation pair composed of an orthogonal transform and a diagonal scaling matrix for each fully connected layer. Unlike SpinQuant (Liu et al., 2025b), this approach jointly optimizes scaling and rotation in a layer-wise manner, enabling comprehensive distribution refinement across activations, weights, and KV caches. As a result, OSTQuant achieves higher accuracy than prior work under the W4A4KV4 configuration.

4.1.2. QAT

QAT incorporates quantization operations directly into the training process, enabling models to retain high accuracy even under low-bit precision. However, because QAT involves jointly optimizing quantization parameters across the entire model, it incurs substantial computational and memory overhead—making it impractical for extremely large models such as GPT3-175B. To address this challenge, recent research has explored various strategies to reduce the cost of QAT, including data-free distillation, block- and group-wise quantization, and even true 1-bit QAT by replacing standard transformer linear layers with BitLinear modules.

LLM-QAT (Liu et al., 2024c) was proposed to address the performance degradation of conventional PTQ methods in sub-8-bit settings, representing the first application of QAT to large-scale LLMs. To account for scenarios where training data is inaccessible, it introduces data-free distillation, training the student model on text generated by a pretrained teacher model. A hybrid sampling strategy—deterministically selecting the initial tokens and probabilistically sampling the remaining ones—is combined with logit distillation to enhance generalization. Furthermore, unlike prior approaches, LLM-QAT also quantizes the KV cache, thereby alleviating memory bottlenecks during long-sequence generation. As a result, it demonstrates strong performance even under W4/A8/KV4 quantization, validating the feasibility of QAT-based LLM quantization.

However, LLM-QAT (Liu et al., 2024c) faces significant limitations when directly applied to large-scale models, as it incurs excessively long training times and high GPU memory usage. To address these challenges, EfficientQAT (Chen et al., 2025) introduces a two-stage training procedure consisting of Block-wise Training of All Parameters (Block-AP) and End-to-End Training of Quantization Parameters (E2E-QP). Block-AP independently trains each Transformer block, optimizing all parameters within the block and performing block-wise reconstruction to minimize quantization error. E2E-QP then fixes the quantized weights obtained from Block-AP and learns the step sizes across the entire model in an end-to-end manner, thereby capturing inter-block interactions for fine-grained optimization. Using this framework, EfficientQAT quantizes the LLaMA2-70B model to 2-bit precision in just 41 hours on a single A100-80GB GPU—compared to the 900 hours required by LLM-QAT. Furthermore, in the 2-4-bit regime, EfficientQAT achieves higher zero-shot accuracy and lower PPL than prior PTQ methods (e.g., GPTQ, QLLM), demonstrating its practicality as a QAT approach for large-scale LLMs.

BitNet (Wang et al., 2023) is a 1-bit quantized Transformer architecture for LLMs that binarizes weights to ± 1 and quantizes activations to 8 bits, achieving both stable training and strong inference performance. Standard linear layers are replaced with BitLinear layers, which compute using 1-bit weights obtained by applying a sign function to the deviation of each weight from its mean, together with activations quantized via an absmax-based 8-bit scheme. During training, non-differentiable operations are addressed using the straight-through estimator (STE), while gradients and optimizer states are preserved in full precision. As a result, BitNet achieves lower PPL and higher accuracy compared to various PTQ methods, and as model size increases, the performance gap relative to FP16 narrows, consistent with scaling laws. Nevertheless, the restricted binary range of ± 1 imposes a limitation on the model's representational capacity.

To overcome this limitation, 1.58 Bits (Ma et al., 2024) extends the weight representation to the ternary set $-1, 0, +1$ and introduces an absmean-based quantization function, enabling a fully integer-based computation structure without multiplications. Specifically, the weight matrix is normalized by its mean absolute value, and each element is rounded and clipped to the nearest ternary value, resulting in integer-only weights that allow most inference operations to be carried out using efficient integer additions. Built on the LLaMA architecture (Touvron et al., 2023a,b), the method further enhances computational efficiency by eliminating all bias terms. Consequently, for models exceeding 3B parameters, 1.58 Bits maintains PPL and end-task accuracy comparable to FP16 LLaMA, while reducing memory usage by up to $7.16\times$ and improving inference speed by up to $4.1\times$.

BitNet (Wang et al., 2023) and 1.58-bit quantization (Ma et al., 2024) demonstrated that stable training is feasible at extremely low precision, thereby opening the possibility of binary LLMs. Nonetheless, these approaches still exhibit limited representational capacity and noticeable accuracy degradation compared with full-precision models. To mitigate these limitations, OneBit (Xu et al., 2024b) introduces a QAT-based method built on a pretrained full-precision model, in contrast to BitNet's training-from-scratch paradigm. By combining 1-bit QAT training, the straight-through estimator, and binary-adapted optimization strategies, OneBit enhances convergence stability and expressive power. Consequently, it achieves PPL levels competitive with FP16 even under a 1-bit setting, substantially improving the practical viability of binary LLMs.

However, OneBit uniformly binarizes all weights to 1-bit, preventing effective exploitation of structural redundancy within the model and neglecting layer-wise variations in parameter importance that could benefit from adaptive sparsity. STBLLM (Dong et al., 2025) addresses these shortcomings by showing that the inherent redundancy among binary weights can be systematically removed and by integrating the standard N:M semi-structured sparsity pattern into binary quantization, thereby enabling sub-1-bit representation. This framework achieves an ultra-low

precision level of 0.55 bits per weight and, on the LLaMA-65B model, attains a practical PPL of 11.07 on the WikiText-2 dataset under a 0.55-bit quantization configuration.

From a broader perspective, Table 1 reveals a consistent set of trade-offs between accuracy, bit-width, and deployment cost across PTQ and QAT regimes. In the moderate 4-6-bit regime, PTQ methods that explicitly correct for outliers in weights and activations (e.g., GPTQ (Frantar et al., 2022), AWQ (Lin et al., 2024), SmoothQuant (Xiao et al., 2023)) are widely adopted. More recently, SpinQuant (Liu et al., 2025b) and OSTQuant (Hu et al., 2025b) further extend this regime by enabling integrated W4-A4-KV4 quantization, achieving accuracy close to full precision while remaining particularly attractive for environments that demand efficient long-context processing. As the bit-width is pushed to 2–3 bits, more structured PTQ schemes such as QuIP# (Tseng et al., 2024), AQLM (Egiazarian et al., 2024), and LeanQuant (Zhang & Shrivastava, 2025) trade some PPL and zero-shot accuracy for substantially higher compression, with AQLM achieving a more favorable accuracy-compression balance on LLaMA2-7B, whereas LeanQuant excels in memory footprint and compatibility with very large models (e.g., LLaMA3.1-405B). In contrast, QAT-based approaches (e.g., LLM-QAT Liu et al., 2024c, EfficientQAT Chen et al., 2025) demand significantly higher training cost but achieve accuracy comparable to or better than classical PTQ baselines (e.g., GPTQ, QLLM Liu et al., 2023) in the ultra-low 2-4-bit regime, especially when data or distillation signals are available; architecture-level designs such as BitNet (Wang et al., 2023), 1.58 Bits (Ma et al., 2024), OneBit (Xu et al., 2024b), and STBLLM (Dong et al., 2025) further push precision toward the 1-0.55-bit range at the price of more intrusive changes and, for the most aggressive settings (e.g., sub-1-bit STBLLM), non-trivial degradation on some benchmarks. Practically, this suggests a tiered design choice: when strict accuracy constraints or long-context processing demands are prioritized, 4-6-bit weight-activation PTQ methods that additionally support KV-cache quantization (e.g., SpinQuant, OSTQuant) are particularly suitable; when memory is the primary bottleneck and modest accuracy loss is acceptable, 2-3-bit weight-only PTQ with learned codebooks or loss-aware grids (e.g., AQLM, LeanQuant) offers the best compression; and in highly resource-constrained or hardware-specialized settings where integer- or bit-level compute is critical, QAT-based and binary/ternary architectures become the methods of choice despite their higher training and implementation complexity.

4.2. Pruning

Pruning is a representative technique that induces sparsity by setting less important weights or activations to zero, which simultaneously reduces the number of parameters, memory bandwidth, and computational cost. Pruning methods are broadly categorized into two types, and Table 2 summarizes and compares recent pruning method on LLaMA2-7B by presenting their pruning scope, criteria, compression ratios, and averaged zero-shot accuracies across common-sense reasoning benchmarks (*i.e.*, WinoGrande and HellaSwag). Weight pruning statically removes trained weights to compress the model size, while activation pruning dynamically zeros out small-magnitude or less-important activations during inference to reduce memory I/O and computational load.

4.2.1. Weight pruning

Weight pruning is a representative compression technique that improves efficiency by removing redundant parameters from the model. The goal is to reduce model size and computation while maintaining prediction accuracy. This requires identifying which weights or structural units to remove and, if necessary, adjusting the remaining weights to compensate for potential quality degradation. Based on the granularity of the removal, this method is classified into structured pruning and unstructured pruning.

Structured pruning removes unnecessary components in deep learning models at the level of structural units such as neurons, channels, at-

tention heads, or layers. By eliminating entire blocks or compute units that contain low-magnitude or low-importance weights, it not only reduces computation but also preserves a simpler computational structure.

SliceGPT (Ashkboos et al., 2024) is a pruning method proposed to overcome the limitations of large-scale language model pruning, where existing approaches often suffer from accuracy degradation, require additional fine-tuning, and yield only limited speedups, making them less scalable in practice. SliceGPT exploits the rotational invariance of RMSNorm to rotate weight matrices into a PCA-based orthogonal basis, where directions with low information variance can be identified and pruned to achieve structured sparsity. Furthermore, both MLP and attention modules are divided into slices, and the contribution of each slice is quantified through RMSNorm; slices with lower importance are pruned, while the removed information is compensated by a correction transformation $Q^T Q$. SliceGPT maintains model performance without fine-tuning while simplifying the model structure and enhancing hardware efficiency. Experimental results show that SliceGPT prunes 30% of parameters in LLaMA2-70B, OPT-66B, and Phi2 (Jawaheripi et al., 2023) while preserving over 90% of zero-shot task accuracy, and achieves up to 36% inference speedup in terms of GPU memory usage without additional optimizations, while still retaining 84% of the dense model performance at 30% pruning on LLaMA2-7B in our benchmark.

LLM-pruner (Ma et al., 2023) is a pruning method that operates on a local structural scope—such as neurons, channels, or layers—using gradient-based importance estimation. The importance of each structure is measured using a loss-based metric derived from the gradient magnitude, and pruning is conducted in a one-shot manner by progressively removing the least important groups. Neuron importance is evaluated by the sum of gradients multiplied by the associated weights, channel importance by the aggregated weight importance, and layer importance by the loss change when removed. After pruning, performance is quickly restored using Parameter-Efficient Fine-Tuning (PEFT). In practice, applying this method to LLaMA, Vicuna, and ChatGLM (GLM et al., 2024) demonstrated that even after pruning 20% of parameters in LLaMA-7B, 94.97% of the original performance was retained, and the entire pruning and post-tuning process was completed in under 3 hours.

FASP (Hu et al., 2025a) is a structured pruning method that reduces redundant computation by jointly removing specific columns from a downstream layer (W_2) and the corresponding rows from the preceding layer (W_1). Pruning decisions are made using a magnitude-based importance metric defined as the product of weight magnitude and the ℓ_2 -norms of the associated input activations. The pruning mask is generated in a one-shot manner without complex iterative optimization. To restore the model, FASP applies a fast least-squares recovery method that adjusts the remaining weights using a single matrix inverse, minimizing accuracy degradation without requiring retraining. Pruning is performed in a global scope that considers inter-layer structural connectivity, preserving over 92% of the dense performance at 20% sparsity in our benchmark results.

SlimLLM (Guo et al., 2025) is proposed to address the limitations of conventional structured pruning methods for LLMs, which primarily rely on weight magnitude and therefore fail to capture vector-direction properties of channels and heads or differences in redundancy across layers. In multi-head attention, SlimLLM measures the change in the final output when a head is removed using the Pearson correlation coefficient, and then applies a greedy search to select the head combination that best preserves the representation, effectively accounting for interactions between heads. For FFN pruning, SlimLLM applies PCA to the covariance of output activations and evaluates down-projection weights in this feature space, reflecting both their alignment with important principal directions and the magnitude of input activations, enabling semantically meaningful channel selection beyond simple magnitude-based pruning. Experimental results show that SlimLLM achieves up to a three percent accuracy improvement and up to a 25% PPL reduction over LLM-Pruner and LoRAP-based methods under structured pruning on LLaMA and LLaMA2, while reducing parameter counts and inference

Table 2

Comparison of pruning methods for LLaMA2-7B: compression ratio and 0-shot accuracy.

Method	Unit	Criterion	I/O	FT	Scope	Ratio	0-shot
Ashkboos et al. (2024)	Struct.	Act.	OS	No	L	30%	56.50 → 47.26
Ma et al. (2023)	Struct.	Loss	OS	PEFT	L	20%	70.00 → 66.61*
Hu et al. (2025a)	Struct.	Magnitude	OS	No	G	20%	73.16 → 67.68*
Guo et al. (2025)	Struct.	Act.	It.	No	L	50%	70.00 → 57.87*
Sun et al. (2024a)	Unstruct.	Magnitude	OS	No	L	50%	63.04 → 60.34
Bai et al. (2024b)	Unstruct.	Loss	It.	No	G	90%	63.43 → 38.98
Liu et al. (2025a)	Unstruct.	Regular	It.	No	G	50%	63.14 → 62.59
Chua et al. (2024)	Static	Act.	OS	No	L	65%	63.15 → 59.25
Song et al. (2025)	Static	Regular	It.	Yes	L	90%	71.42 → 67.55
Liang et al. (2025)	Static	Loss	OS	No	L	50%	72.58 → 59.04
Zhang et al. (2025c)	Static	Act.	OS	No	G	50%	63.13 → 60.83
Le et al. (2025)	Dynamic	Loss	It.	No	L	40%	71.35 → 60.55

Unit: Struct. = Structured, Unstruct. = Unstructured, Static = offline mask fixed at inference, Dynamic = input-dependent.

Criterion: Loss = Loss-based, Regular = Regularization-based, Magnitude = Magnitude-based, Act. = Activation-based.

I/O: It. = Iterative, OS = One-shot, N = Neither. **Scope:** L = Local, G = Global.

0-shot: The average 0-shot accuracy over two benchmarks (HellaSwag and WinoGrande).

PEFT: Parameter-Efficient Fine-Tuning.

*: denotes results obtained on LLaMA-7B

latency to deliver over a 30% speedup on GPU, preserving about 83% of its dense performance even at 50% pruning in our benchmark results.

In contrast, unstructured pruning eliminates individual low-importance weights to induce sparsity. This method allows fine-grained control without structural constraints, enabling selective removal of connections that contribute less to performance. However, the resulting irregular sparsity patterns pose challenges for optimization on parallel hardware architectures such as GPUs.

Previous LLM pruning methods often rely on retraining, complex weight reconstruction procedures, or simple magnitude-based criteria, which makes them difficult to apply to large-scale models and leads to significant performance degradation. Wanda (Sun et al., 2024a) introduces a simple importance metric—the product of weight magnitude and the ℓ_2 -norm of its input activation—enabling efficient one-pass pruning without retraining and preserving frequently activated weights over indiscriminate magnitude pruning. This method is performed in just one forward pass and prunes weights locally with respect to output neurons, which distinguishes it from purely global approaches. Experimental results demonstrate that Wanda outperforms conventional magnitude pruning and achieves performance comparable to SparseGPT (Frantar & Alistarh, 2023) on LLaMA models for WikiText and C4, while retaining about 96% of the original performance even at 50% sparsity in our benchmark results.

SparseLLM (Bai et al., 2024b) is a global unstructured pruning framework that removes weights element-wise while considering inter-layer dependencies. It decomposes the overall pruning problem into subproblems and introduces auxiliary variables with soft constraints to maintain GPU efficiency during layer-aware optimization. While FFN layers are pruned globally, MHA layers are pruned locally, with an iterative loss-based metric refining the pruning mask throughout the process. Without requiring post-pruning fine-tuning, SparseLLM achieves up to 80% lower PPL than previous sota methods, even at over 60% sparsity on WikiText-2, establishing itself as a global pruning alternative to SparseGPT (Frantar & Alistarh, 2023) and Wanda (Sun et al., 2024a), although our benchmark shows that 90% sparsity retains only about 61% of the original performance.

OPTISHEAR (Liu et al., 2025a) is a post-training sparsification method designed to overcome the limitations of prior pruning techniques (Sun et al., 2024b), which rely on fixed heuristic metrics and exhibit significant performance degradation when applied to large models with distributional shifts such as LLaMA3. OPTISHEAR introduces a regularization-based meta pruning metric that integrates both weight

magnitude and the ℓ_2 -norm of input activations through a functional combination with trainable coefficients. It further leverages the Frobenius norm of output reconstruction error along with NSGA-III optimization to iteratively balance accuracy, model size, and computational cost. After optimization, the pruning mask is fixed and can be directly applied without fine-tuning. While OPTISHEAR performs element-wise removal, it computes importance scores at the global level and thus belongs to global unstructured pruning. By combining weight magnitude and activation information within the meta pruning metric and optimizing through NSGA-III, OPTISHEAR automatically adjusts the trade-off between accuracy and model size while minimizing reconstruction error, retaining over 99% of the original performance even at 50% sparsity in our benchmark.

4.2.2. Activation pruning

Activation pruning is a sparsification technique that removes less important intermediate activations generated during a model's inference process. In architectures such as Transformers, the attention score matrix and the intermediate activations in the FFN module are particularly large, so applying sparsity to these components can significantly reduce memory usage and computational cost during inference. Activation pruning is broadly categorized into static and dynamic approaches, depending on when it is applied and how the pruning mask is generated.

Static activation pruning is a sparsification method that disables low-importance neurons or activation positions based on prior analysis. By using a predetermined pruning pattern, memory and computational resources can be predicted in advance. This property also enables easier hardware-level parallelization and optimization.

SCAP (Chua et al., 2024) is proposed to overcome the limitations of previous approaches that relied on additional retraining or complex predictor learning. In particular, when the activation distribution is not concentrated near zero, making effective removal difficult with a simple threshold, SCAP introduces a mode-centering technique. This technique estimates the mode of the distribution using histogram-based statistics and aligns it to the origin, thereby maximizing the effectiveness of L1-based pruning. The procedure can be performed with only a small amount of calibration data, and once the mask is determined, it remains fixed regardless of runtime inputs, resulting in a static pruning structure. Experimental results show that SCAP removes up to 48.5% of FFN channels in Mistral-7B while maintaining accuracy within 1.5% of the original, and achieves 1.5× faster decoding speed compared to

CATS (Lee et al., 2024) at the same accuracy level, while retaining over 94% of dense performance even at 65% sparsity in our benchmark.

ProSparse (Song et al., 2025) is a static activation pruning method that maximizes FFN activation sparsity to improve inference efficiency. By replacing dense-output activation functions like GELU or Swish with ReLU, ProSparse inherently zeros out negative inputs, deactivating numerous neurons. The method further enhances sparsity through progressive ℓ_1 -norm regularization and activation threshold shifting, which gradually moves the ReLU threshold into the positive domain during training. Once established early in training, the sparse subnetwork and its fixed pruning mask are maintained throughout, eliminating the need for fine-tuning. Operating locally within FFN layers based on neuron activations, ProSparse achieves up to 89.32% activation sparsity on LLaMA2-7B, LLaMA2-13B, and MiniCPM-1B, while preserving accuracy comparable to dense Swish-based models, with inference speedups reaching 4.52 $\times$ on PowerInfer. In benchmark, ProSparse consistently maintained accuracy, retaining over 95% of the dense performance even at nearly 90% sparsity, further validating its robustness under extreme pruning.

SEAP (Liang et al., 2025) is a static activation pruning technique that performs task-adaptive neuron selection in pretrained LLMs without fine-tuning. It statistically analyzes the activation distribution of hidden states for each input to select important neurons in a one-shot manner for each layer. The pruning mask is fixed without gradients and applied at the layer-local level, removing only the unnecessary neurons in the forward path to reduce computation. Experimental results show that SEAP achieves 97.8% of the dense model's average accuracy on zero-shot tasks such as BoolQ, ARC, HellaSwag, OBQA, PiQA, and Winogrande with 20% pruning on LLaMA2-7B and LLaMA2-13B, and improves inference speed by up to 1.52 $\times$ on LLaMA2-13B. Moreover, in our benchmark, SEAP retains over 81% of the dense performance even at 50% sparsity, confirming its robustness under aggressive pruning.

R-Sparse (Zhang et al., 2025c) is an activation-based sparsity technique that requires no additional training and addresses the issue that conventional approaches achieve low sparsity and suffer large performance drops when using nonlinear functions such as GELU and SiLU. The method evaluates activation magnitude and low-rank contribution using a small amount of inference data, selects the least important channels according to a fixed threshold, and stores them as a static channel mask that is applied directly during execution. Instead of zeroing out small activations, it approximates them with constant values absorbed into biases to minimize information loss, and it decomposes weights into a low-rank structure that preserves key components while combining the remaining parts with a sparse computation path to reduce cost. Experimental results show that R-Sparse applies 50% sparsity to LLaMA and Mistral models without training and keeps accuracy loss within 3%, achieving an average accuracy gain of about 18% compared to previous activation pruning. On LLaMA2-7B, it retains more than 98% of the original performance and delivers up to 42% GPU inference speedup, which aligns with our benchmark result showing over 97% retention even at 50% sparsity.

Dynamic activation pruning removes low-importance activations at runtime, depending on the specific input being processed. Unlike static methods, the pruning mask in dynamic pruning is generated on-the-fly, making the pruning pattern input-dependent. This dynamic nature introduces irregular computation paths, which can make hardware-level optimization difficult and add runtime overhead, since importance must be evaluated in real time.

Sparse Flash Attention (Pagliardini et al., 2023) employs two dynamic pruning strategies at the token/head level. First, **QK** dropping discards low-importance Q or K vectors. Second, hash-based grouping uses Locality-Sensitive Hashing (LSH) to cluster tokens with similar embeddings into buckets, restricting attention computation within each bucket. As these methods rely on similarity grouping rather than explicit importance metrics, Sparse Flash Attention is classified as metric-free pruning and requires no fine-tuning. On OpenWebText2, it accel-

erates training by 2.0 $\times$ and 3.3 $\times$ for sequence lengths of $T = 8, 192$ and $T = 16, 384$ respectively, while achieving up to 3 $\times$ faster inference and 50% FLOPs reduction compared to standard FlashAttention (Dao et al., 2022).

Previous LLM pruning methods rely on fixed calibration data and single importance metrics, which fail to capture batch-specific dynamic outliers and thus suffer from performance degradation and additional fine-tuning overhead. PROBE PRUNING (Le et al., 2025) addresses this limitation by dynamically removing less important FFN channels in a batch-wise structured manner. Specifically, only 1.5% of the total computation is used to lightly execute a subset of the input batch and obtain probe results, which are then integrated with the historical hidden states from the calibration dataset using an importance-scaled aggregation to compute channel-wise pruning scores. These scores are iteratively updated for every batch and applied to structured pruning at the FFN channel level. The pruning mask is dynamically generated according to the runtime input, enabling automatic adjustment of sparsity. As a result, PROBE PRUNING can be applied without additional modules or fine-tuning while maintaining efficiency, preserving over 84% of the dense model's performance even at 40% sparsity in our benchmark.

Structured weight pruning methods (e.g., SliceGPT Ashkboos et al., 2024, LLM-Pruner Ma et al., 2023, FASP Hu et al., 2025a, SlimLLM Guo et al., 2025) remove structured units such as slices, heads, or channels, so sparsity translates directly into FLOP and memory reductions; in our benchmark they retain about 83–95% of the dense performance at 20–50% sparsity while preserving a regular computation pattern that is amenable to hardware optimization. In contrast, global unstructured pruning methods (e.g., SparseLLM Bai et al., 2024b, Wanda Sun et al., 2024a, OPTISHEAR Liu et al., 2025a) prune individual elements, which allows pushing sparsity more aggressively beyond 60% at the cost of more irregular sparsity patterns. Static activation pruning methods (e.g., SCAP Chua et al., 2024, SEAP Liang et al., 2025, R-Sparse Zhang et al., 2025c, and ProSparse Song et al., 2025, which all employ fixed pruning masks in our description) simplify the compute graph while exploiting FFN activation sparsity to obtain substantial speedups and reduced activation I/O. Dynamic activation pruning methods (e.g., Sparse Flash Attention (Pagliardini et al., 2023), PROBE PRUNING (Le et al., 2025)), on the other hand, adapt their masks to each input and achieve significant efficiency gains, particularly for long sequences, by tracking input-dependent sparsity at runtime. Overall, our benchmark suggests that structured pruning with activation-aware criteria is effective when both accuracy retention and predictable hardware speedups are important, global unstructured pruning with activation-informed meta metrics (e.g., OPTISHEAR) can offer a favorable accuracy–compression trade-off at moderate sparsity levels, and dynamic activation pruning is especially beneficial for scenarios with long contexts and input-dependent sparsity patterns.

4.3. Knowledge distillation

Knowledge distillation (KD) is a widely adopted technique that transfers knowledge from a teacher model to a student model to improve inference efficiency while preserving model performance. Early KD methods primarily relied on mimicking the teacher's output distribution (logits), but recent approaches have evolved into more sophisticated forms that incorporate richer supervisory signals such as hidden states and rationales derived from intermediate reasoning steps. KD methods are typically categorized into white-box, black-box, and grey-box settings based on the level of access to the teacher model's internal representations. In white-box settings, the student has direct access to intermediate features such as logits and hidden states. In contrast, commercial API-based models like GPT-4 fall under black-box settings, where only final outputs are observable. To bridge this limitation, grey-box strategies have been proposed, often leveraging proxy teacher models to approximate internal guidance. KD can be applied during pretraining, fine-tuning, or across both stages (Liang et al., 2023). While KL divergence remains

Table 3

Comparison of knowledge distillation methods with various models and benchmarks.

Method	Box	Signal	Stage	Loss	Teacher / Student	Score – Benchmark
Timiryasov and Tastet (2023)	W	L	PT	KL + CE	GPT2-705M + LLaMA-360M / LLaMA-58M	68.0 → 69.8 – BLIMP (0-shot)
Liang et al. (2023)	W	L + H	PT/FT	KL + MSE	GPT2-125M / GPT2-82M DeBERTaV3-183M / DeBERTaV3-70M	46.0 → 23.0 – LD* (0-shot) 88.9 → 87.5 – GLUE*
Kim et al. (2023)	W	L	FT	KL	OPT-6.7B / OPT-6.7B (W2A16) LLaMA-7B / LLaMA-7B (W2A16)	10.21 → 11.17 – PTB (PPL) 30.25 → 26.23 – GSM8K (Acc.)
Gu et al. (2024)	W	L	FT	RKL	LLaMA-13B / LLaMA-7B	38.5 → 40.2 – UnNI (Rouge-L)
Ko et al. (2024)	W	L + H	FT	CE + MSE	GPT2-1.5B / GPT2-117M OpenL-LLaMA2-7B / OpenLLaMA2-3B	23.41 → 22.91 – Mix. (Rouge-L) 25.28 → 28.99 – Mix. (Rouge-L)
Zhang et al. (2024)	W	L + H	FT	KL, RKL, JS	GPT2-1.5B / GPT2-120M LLaMA2-7B / TinyLLaMA-1.1B	23.42 → 18.69 – Mix. (Rouge-L) 26.50 → 26.65 – Mix. (Rouge-L)
Du et al. (2024)	W	L	FT	CKL	LLaMA2-7B / 3-bit-Quant. model LLaMA2-7B / 2-bit-Quant. model MetaMath-7B / 2-bit-Quant. model	46.45 → 43.65 – MMLU (5-shot) 46.45 → 29.25 – MMLU (5-shot) 66.41 → 51.02 – GSM8K (Acc.)
Dasgupta and Cohn (2025)	W	H	PT + FT	CKA	mBART-large / mBART-base mBART-large / mBART-base	27.0 → 26.3 – WMT16 (BLEU) 40.0 → 42.3 – IWSLT17 (BLEU)
Xu et al. (2025)	B	L	FT	KL	Gemma-7B / Gemma-2B Qwen2-7B / Qwen2-0.5B	36.8 → 35.0 – DS* (Rouge-L) 73.8 → 36.6 – GSM8K (Acc.)
Huang et al. (2022)	B	L	FT	KL + CE	BERT-340M / BERT-110M GPT2-762M / GPT2-345M	25.6 → 27.2 – LAMA / P@1 66.2 → 65.8 – CrossFit (F1)
Hsieh et al. (2023)	B	R	FT	CE	PaLM-540B / T5-Base-220M PaLM-540B / T5-Base-220M	N/A → 89.51 – e-SNLI (Acc.) N/A → 49.58 – ANLI (Acc.)
Ho et al. (2022)	B	L + R	FT	CE	InstructGPT-175B / GPT3-6.7B InstructGPT-175B / GPT3-6.7B	73.87 → 83.78 – Date* (Acc.) 50.22 → 73.33 – TSO* (Acc.)
Jiang et al. (2023b)	B	R	FT	CE	ChatGPT3.5-turbo / Lion-13B ChatGPT3.5-turbo / Lion-13B	47.2 → 34.2 – AGIEval (0-shot) 48.9 → 36.2 – BB* (0-shot)
Lupart et al. (2025)	B	R	FT	CE	MistralQR / DiSCo Mistral HumanQR / DiSCo Human	0.759 → 0.842 – TO* (R@100) 0.912 → 0.927 – QR* (R@100)
Hamman et al. (2025)	W	L + H	FT	CE + KL + MSE	DeBERTa-v3-base / DeBERTa-v3-small Qwen2.5-1.5B / Qwen2.5-0.5B	93.8 → 86.1 – IMDB (8-shot) 94.3 → 80.0 – IMDB (8-shot)
Chen et al. (2024)	G	L	FT	KL	GPT-4 / LLaMA2-7B GPT-4 / LLaMA2-7B	92.00 → 53.07 – GSM8K (0-shot) 86.40 → 51.35 – MMLU (0-shot)

Box: W = White-box, B = Black-box, G = Grey-box. **Signal:** L = Logits, H = Hidden states, R = Rationale. **Stage:** PT = Pretraining, FT = fine-tuning. **Notation:** We use $a \rightarrow b$ to denote teacher → student scores; larger is better for all metrics except PPL, for which smaller is better.

GLUE* denotes the averaged GLUE score over standard NLU tasks (e.g., MNLI, QQP, QNLI, SST-2, CoLA, STS-B, MRPC, RTE) using the official task-specific metrics.

Mix. denotes the average Rouge-L over five instruction-following datasets: Dolly, SelfInst, VicunaEval, S-NI, and UnNI.

DS*, TO*, LD*, BB*, and QR* denote specific benchmarks: DS* = DialogSum; TO* = TopiOCCA; LD* = LAMBADA; BB* = BIG-Bench; QR* = QReCC. Date* and TSO* denote reasoning benchmarks: Date* = Date Understanding; TSO* = Tracking Shuffled Objects.

Metrics and evaluation settings shown in parentheses (e.g., Acc., F1, BLEU, Rouge-L, PPL, GLUE, R@100, 0/5/8-shot) follow the definitions in the original papers and benchmark suites.

the most widely used loss for KD, recent studies have explored alternative or composite objectives—such as cross-entropy (CE), mean squared error (MSE), reverse KL (RKL), and Jensen–Shannon (JS) divergence—tailored to different distillation targets (Du et al., 2024; Gu et al., 2024). Table 3 provides a taxonomy of representative KD methodologies and, for each method, lists the teacher-student configuration together with the reported accuracy on their respective benchmarks.

4.3.1. White-box knowledge distillation

BabyLLaMA (Timiryasov & Tastet, 2023) trains a student model using an ensemble teacher constructed by averaging the logits of GPT2 and a small LLaMA model. Using this strategy, a compact student model with only 58M parameters was able to outperform its teacher in terms of accuracy. However, mimicking intermediate representations across all layers introduced significant computational overhead. Additionally, due to the capacity gap between the student and teacher, the student model struggled to capture rare high-dimensional modes—complex contextual patterns that occur infrequently and require substantial representational power.

To mitigate this limitation, TED (Liang et al., 2023) inserts lightweight filters into each layer to selectively distill only those channels that are relevant to downstream tasks, thereby reducing both computational cost and representational noise.

To alleviate the problem of accumulated quantization error, TSLD (Kim et al., 2023) introduces a loss reweighting mechanism based

on the teacher model’s cross-entropy scores—assigning smaller penalties to high-confidence samples and larger penalties to low-confidence ones. This strategy enables ternary models to maintain accuracy levels comparable to their FP32 counterparts.

The conventional KL-based loss function often produces excessive gradients for low-importance tokens. To mitigate this, MiniLLM (Gu et al., 2024) replaces the loss with reverse KL divergence, encouraging the student model to focus more on tokens to which the teacher assigns high confidence. This approach yields consistent performance improvements across student models ranging from 120M to 13B parameters. Notably, experiments using LLaMA-7B and LLaMA-13B as teachers showed an average accuracy gain of 1.7–4.2 percentage points over standard KD baselines.

During inference, the student model operates in an on-policy setting—conditioning on its previously generated tokens—while the teacher conditions on ground-truth tokens, leading to a distribution mismatch. GKD (Agarwal et al., 2023) addresses this by comparing the teacher and student over student-generated sequences, and updating the student via a combined loss that integrates teacher-logit KL divergence, cross-entropy, and reinforcement learning objectives. In numerical reasoning experiments on GSM8K, a student trained with supervised fine-tuning achieved 34.8% accuracy, whereas GKD improved this to 41.2%, yielding a gain of 6.4 percentage points. To further improve efficiency and mitigate the high cost of generation, one can also employ a combination of off-policy and mixed schedulers.

DistiLLM (Ko et al., 2024) introduces Skewed KL divergence, a variant of KL loss with temperature scaling, which encourages the student to concentrate on high-probability (i.e., important) tokens while suppressing gradient explosion from low-probability ones. In addition, it employs adaptive off-policy training, which periodically re-injects the teacher's outputs to recalibrate the target distribution and accelerate convergence.

When the teacher and student models use different tokenizers, the resulting token sequences can be misaligned, leading to mismatches in output dimensionality. DSKD (Zhang et al., 2024) resolves this issue by first linearly projecting the hidden states into a shared dimensional space and aligning them using an ℓ_2 -loss. It then applies cross-model attention to refine sequence alignment and transforms the teacher's logits into the student's vocabulary space, enabling the use of KL loss for probability space alignment. In experiments, distilling Qwen1.5-1.8B into GPT2-120M achieved a 1.66 percentage point improvement in ROUGE-L, while distilling Mistral-7B into TinyLLaMA-1.1B led to a 5.48 percentage point gain. These results demonstrate that DSKD (Zhang et al., 2024) consistently outperforms standard sequence-level KD approaches.

BitDistiller (Du et al., 2024) mitigates accuracy degradation in low-bit quantization settings by defining token-wise confidence as the maximum value of the student model's softmax output and incorporating it as a weighting factor in the KL loss. This strategy, referred to as confidence-aware KL (CKL) divergence, assigns higher penalties to high-confidence tokens and lower penalties to low-confidence ones. In experiments, BitDistiller trains a student model quantized to 2-bit weights and 16-bit activations using LLaMA2-13B as the backbone, and achieves a PPL increase of just 1.9 compared to the full-precision model on the WikiText-2 dataset, demonstrating superior accuracy retention over previous approaches. Key findings remain empirical, and explaining why the 13B teacher does not consistently outperform the 7B model, along with extending the approach to 1-bit and vector quantization, are left for future work.

Recent work (Dasgupta & Cohn, 2025) introduces a white-box distillation method that addresses the limitations of cosine similarity-based representation alignment by employing Centered Kernel Alignment (CKA) to align the structural properties of hidden states, even when the teacher and student models differ in dimensionality. This approach achieves performance comparable to traditional distillation techniques across diverse architectures and tasks—including BERT, BART, mBART, and T5—while offering several advantages: it supports smaller student models, enables higher compression ratios, and incurs minimal training and inference overhead. The study primarily focuses on encoder and encoder-decoder models such as BART, mBART, Flan-T5, and BERT, while evaluation on decoder-only LLMs remains limited.

Speculative Knowledge Distillation (Xu et al., 2025) performs collaborative generation between the student and teacher to dynamically improve the training data. Concretely, the student proposes the next token, and the teacher replaces low-confidence tokens in real time to inject high-quality guidance. Through this on-policy distillation, the student receives feedback aligned with its own generation distribution, consistently outperforming existing KD approaches across diverse generative tasks such as machine translation, summarization, and mathematical reasoning.

4.3.2. Black-box knowledge distillation

While previous methods are designed for white-box scenarios, the increasing dominance of commercial models such as ChatGPT and GPT-4—where internal parameters are inaccessible—has made black-box KD increasingly relevant. An early example is Multitask In-Context Tuning (Huang et al., 2022), which constructs a single prompt by concatenating few-shot exemplars from multiple tasks and feeds it to the teacher model. The resulting logit distribution is recorded as soft labels and used to supervise the student model, in conjunction with a language modeling loss, during training.

Subsequent research has moved beyond simple answer imitation toward distilling the reasoning process itself. Distilling Step-by-Step (Hsieh et al., 2023) incorporates the chain-of-thought (CoT) reasoning paradigm, training the student model to replicate intermediate inference steps in order to mitigate error propagation. Experiments on arithmetic and reasoning benchmarks such as GSM8K demonstrate accuracy improvements of up to 7 percentage points.

Fine-tune-CoT (Ho et al., 2022) constructs a large-scale dataset of problem-reasoning-answer triplets by prompting the teacher model with Zero-shot-CoT and filtering for sequences that yield correct answers. To promote data diversity and enhance training stability, multiple CoT paths are sampled per question using temperature sampling. The resulting student model demonstrates superior performance over prompt-based CoT and standard fine-tuning in complex reasoning tasks, and even exceeds the accuracy of the 175B teacher model on certain benchmarks.

Standard knowledge distillation typically follows a unidirectional setup, where the student passively imitates the teacher's responses. This structure often limits the student's ability to effectively learn from challenging cases. To overcome this limitation, Lion (Jiang et al., 2023b) introduces an adversarial knowledge distillation framework in which a single model jointly serves as teacher, referee, and generator. The teacher generates responses, the referee identifies difficult instructions by evaluating discrepancies between teacher and student outputs, and the generator produces hard training examples based on these instructions. The student first learns by imitating the teacher and is then further trained on the generated challenging data. Remarkably, Lion-13B achieves ChatGPT-level response quality with just 70K training samples, and even outperforms Vicuna-13B. However, its maximum sequence length is limited to 1024 tokens.

DiSCo (Lupart et al., 2025) is a black-box distillation method that trains a student retriever to capture semantic similarity with documents by leveraging query rewritings generated by LLM-based teachers. By learning from diverse rewritten queries produced by various teacher models, the student develops more generalized and robust retrieval representations. Experimental results demonstrate that DiSCo consistently outperforms existing dense retrievers across a wide range of open-domain and conversational retrieval benchmarks, including MS MARCO, TopiQA, and QReCC. Future extensions could explore combining stronger LLMs with diverse teacher models, including cross-encoders, to further enhance distillation.

Counterfactual-Explanation-infused Distillation (CoD) (Hamman et al., 2025) improves student model performance in few-shot settings by exploiting counterfactual examples (CFEs), which are inputs that can reverse the teacher model's prediction with only minimal perturbations. By incorporating these CFEs alongside the original samples during training, CoD enables the student model to more accurately capture the fine-grained decision boundaries of the teacher. Experiments across diverse datasets and LLM configurations show that CoD consistently outperforms standard KD methods with as few as 8–512 training samples, and it achieves comparable or superior performance even when using only half of the baseline data.

To address the limitation of lacking access to internal distributions in black-box teachers, Proxy-KD (Chen et al., 2024) trains a LLaMA2-70B proxy teacher solely from GPT-4's response texts. The student is then trained using soft labels from the proxy teacher and hard labels from GPT-4, with a weighted KL divergence loss applied based on the similarity between the two responses to improve distribution alignment. This approach constitutes a grey-box framework: acting as white-box in the proxy region where logits are available, and as black-box in the GPT-4 region where only final outputs are accessible. Experimental results show that an LLaMA2-7B student trained with Proxy-KD outperforms Alpaca-style training by an average of 3–4 points on benchmarks such as MMLU, GSM8K, and HumanEval, demonstrating the method's effectiveness in transferring distributional knowledge. However, the ex-

periments are confined to LLaMA models, and the use of proxy alignment and preference optimization results in prolonged training times.

In summary, existing studies reveal that KD methods span a rich design space along the axes of access level (*i.e.*, white-, black-, and grey-box), supervision signal (*i.e.*, logits, hidden states, and rationales), and computational/memory cost. White-box approaches such as BabyLLaMA (Timiryasov & Tastet, 2023), MiniLLM (Gu et al., 2024), DSKD (Zhang et al., 2024), BitDistiller (Du et al., 2024), and CKA-based representation alignment (Dasgupta & Cohn, 2025) either consistently improve student performance over simple logit matching (*e.g.*, MiniLLM Gu et al., 2024, DSKD Zhang et al., 2024, BitDistiller Du et al., 2024) or maintain performance comparable to standard distillation while enabling smaller students with higher compression ratios and low overhead (*e.g.*, CKA-based methods). In particular, BabyLLaMA (Timiryasov & Tastet, 2023) shows that a student with only tens of millions of parameters can outperform a larger teacher, and TSLD (Kim et al., 2023) and BitDistiller (Du et al., 2024) report that, in ternary and low-bit quantization settings, the quantized models incur only minor degradation relative to their full-precision counterparts. GKD (Agarwal et al., 2023) and Speculative KD (Xu et al., 2025) further mitigate teacher-student distribution mismatch through on-policy distillation, where the student is trained on its own generated trajectories. Meanwhile, black-box methods such as Distilling Step-by-Step (Hsieh et al., 2023), Fine-tune-CoT (Ho et al., 2022), Lion (Jiang et al., 2023b), DiSCo (Lupart et al., 2025), and CoD (Hamman et al., 2025) leverage chain-of-thought rationales, query rewriting, or counterfactual examples to obtain substantial gains over answer-only imitation (*e.g.*, up to a 7-point improvement on GSM8K, or outperforming standard KD with as few as 8–512 training samples). Proxy-KD (Chen et al., 2024) adopts a grey-box strategy by first training a LLaMA2-70B proxy teacher solely on GPT-4 responses and then distilling a LLaMA2-7B student, achieving average improvements of 3–4 points over Alpaca-style training on MMLU, GSM8K, and HumanEval. Overall, these results suggest that, in white-box settings, carefully designed logit/representation-based KD and quantization-aware training are effective for simultaneously achieving high compression ratios and strong performance retention, whereas in black- and grey-box settings, CoT-, rationale-, and proxy-based approaches enable knowledge transfer from powerful proprietary LLMs to compact students using limited data and restricted interfaces.

4.4. Approximation

Approximation encompasses a diverse range of techniques designed to minimize performance degradation while reducing computational resource consumption by substituting or transforming the core computational processes of the original model into more efficient forms. This section reviews research trends by focusing on the mathematical properties of internal operations in LLMs, categorizing them into two primary approaches: optimization methods for linear transformations associated with weight matrices, and simplification strategies for computationally expensive non-linear components such as activation and normalization functions.

4.4.1. Linear approximation

Linear approximation focuses on optimizing linear transformations associated with weight matrices, which constitute the majority of internal operations in LLMs. The most representative approach is low-rank approximation, which decomposes a matrix W into the product of two smaller matrices U and V such that $W \approx UV$, thereby reducing the number of parameters and computational complexity while minimizing information loss. Here, the dimensions of U and V are $m \times k$ and $k \times n$, respectively, where k is the rank of the approximated matrix. As the value of k decreases, the parameter compression ratio increases. While singular value decomposition (SVD) provides the foundation for such approximations, it is limited by its sole focus on minimizing matrix reconstruction error, which can diverge from the goal of preserving task

performance in LLMs. Consequently, various SVD-based approximation techniques that account for the specific characteristics of LLMs have been proposed.

FWSVD (Hsu et al., 2022) points out that the approach of uniformly reconstructing all parameters does not necessarily translate to model performance. To address this, FWSVD estimates the importance of each parameter's contribution to the model's task loss using Fisher information. This method of using Fisher information for parameter importance measurement has been demonstrated to be effective in previous studies (Kirkpatrick et al., 2017; Liu et al., 2021). FWSVD utilizes this importance information as scaling coefficients, multiplying each row of the original parameter matrix by these coefficients before applying SVD. Through this process, parameters crucial to model performance are prioritized for preservation during compression. When applied to DistilBERT, a lightweight BERT-based model, FWSVD achieved approximately 30% parameter reduction on MRPC while effectively preserving performance.

ASVD (Yuan et al., 2023) identifies the limitations of standard SVD for overlooking the activation distributions of LLMs and the varying compression sensitivities across layers. To address this issue, ASVD employs a calibration dataset to measure average activation magnitudes and constructs a diagonal matrix S , subsequently transforming the weights to WS before applying SVD to mitigate the impact of outliers. Furthermore, by applying various SVD ranks to each layer and measuring the overall model performance changes, ASVD determines optimal layer-specific compression ratios. When applied to various sizes of LLaMA2 model families without retraining, ASVD achieved 10–30% parameter compression while maintaining performance on WikiText-2 (Merity et al., 2016b) and The Penn Treebank (Marcus et al., 1993).

LoRD (Kaushal et al., 2024) selectively decomposes only layers with high compression efficiency by analyzing the aspect ratio, rather than decomposing all layers. This approach achieved approximately 23% parameter compression and 22% inference speedup on StarCoder-16B and CodeGen family models without retraining, while maintaining performance comparable to the original models on HumanEval (Chen et al., 2021).

SVD-LLM (Wang et al., 2025b) addresses two fundamental limitations of existing SVD-based approximation techniques: the unclear relationship between compression loss and truncated singular values when truncation is performed solely based on singular value magnitudes, and the inability to effectively compensate for information loss due to compression. To resolve these issues, SVD-LLM applies data whitening that incorporates the statistical properties of input activations, establishing a direct correspondence between truncated singular values and compression loss. Subsequently, it employs sequential LoRA (Hu et al., 2022) to fine-tune the decomposed matrices, effectively recovering lost information. This approach has been applied to various 7B models, including OPT, LLaMA, and Mistral, achieving parameter compression ranging from 16% to 80% while maintaining model performance in most cases. Nevertheless, SVD-LLM suffers from pronounced performance degradation at higher compression ratios and fails to fully account for varying compression sensitivities across modules (*e.g.*, FFN and Attention).

SoLA (Huang et al., 2025) aims at training-free compression, addressing the limitation of SVD-LLM, which requires retraining. It leverages the soft activation sparsity observed in the FFNs of LLMs: a small number of neurons with high norms are preserved as they are, while the remaining low-activation neurons are compressed using SVD. Furthermore, recognizing that the sensitivities of FFN and attention modules differ, SoLA dynamically assigns the optimal rank to each weight matrix by adapting the rank allocation for each component. With this design, SoLA achieves lower PPL and higher downstream accuracy than SVD-LLM without additional training, and demonstrates up to 1.7 $\times$ speedup in inference. Although it achieves superior performance over prior methods without retraining, its focus on neuron- and component-level compression suggests room for improvement, for example, by integrating techniques that measure layer-wise importance.

SVD-LLM V2 (Wang et al., 2025a) aims to further reduce truncation loss and better exploit layer-wise redundancy heterogeneity in post-training SVD-based LLM compression. Rather than repeatedly evaluating PPL over many candidate compression settings, it uses analytically estimated truncation loss to allocate heterogeneous ranks across layers and components, and refines the SVD factorization with a more stable objective that directly minimizes the output discrepancy between the original and compressed model. Across five LLMs and ten benchmarks, SVD-LLM V2 consistently improves PPL and downstream accuracy over FWSVD, ASVD, and SVD-LLM, while keeping compression time to tens of minutes, achieving up to $2.71\times$ higher inference throughput on A100 GPUs, and outperforming structured-pruning and 1-bit PTQ baselines under the same memory budget, especially when paired with low-bit quantization (e.g., GPTQ Frantar et al., 2022).

Taken together, these methods trace a clear evolution in SVD-based linear approximation for LLMs. Early work such as FWSVD (Hsu et al., 2022) and ASVD (Yuan et al., 2023) already show that reconstruction quality alone is not a sufficient target: weighting singular directions by Fisher information or activation statistics and allowing layer-dependent ranks can deliver 10–30% parameter savings on models from DistilBERT to LLaMA2 with little loss on standard language-modeling benchmarks. LoRD (Kaushal et al., 2024) further demonstrates that it is often unnecessary to factorize every layer—selectively decomposing only high-gain blocks yields about 23% parameter compression and 22% inference speedup on StarCoder-16B and CodeGen, while preserving HumanEval accuracy. SVD-LLM (Wang et al., 2025b) and SoLA (Huang et al., 2025) then push compression more aggressively to the 50–80% range by incorporating activation whitening, module-aware sensitivity and, in the case of SVD-LLM, lightweight LoRA finetuning, or, in SoLA, a fully training-free pipeline that still achieves up to $1.7\times$ speedup. SVD-LLM V2 (Wang et al., 2025a) represents a next step toward principled, post-training compression: it uses analytically estimated truncation loss for layer- and component-wise rank allocation and a loss-optimal truncation scheme, consistently improving perplexity and accuracy over prior SVD variants across five LLMs and ten benchmarks, with compression times of only tens of minutes and up to $2.71\times$ throughput gains on A100 GPUs. Overall, these results suggest that successful SVD-based compression hinges less on low-rank factorization itself than on how ranks are adapted to data and layer sensitivity, and on whether small, targeted adaptations are used to recover performance at high compression ratios.

4.4.2. Non-linear approximation

Non-linear approximation constitutes a critical optimization strategy aimed at alleviating computational bottlenecks in neural network inference. This technique specifically targets computationally expensive non-linear operations, such as the softmax function, which often impose significant computational or memory constraints during the inference phase. By replacing these operations with computationally efficient approximations or alternatives, this approach achieves substantial reductions in inference latency, energy consumption, and silicon area requirements, while simultaneously facilitating improved parallel execution efficiency.

Softmax (Stevens et al., 2021) addresses the computational bottlenecks and hardware inefficiencies of softmax operations in LLMs. Softmax employs computationally inexpensive base-2 exponentials and adopts online normalization (Milakov & Gimelshein, 2018) that eliminates the maximum computation step, while enhancing hardware compatibility by utilizing integer-based maximum values and shift-based renormalization that leverages base-2 properties. The dedicated hardware acceleration unit designed based on this algorithm maintains near-identical accuracy to standard softmax on BERT, while achieving $2.35\times$ energy efficiency improvement and 10% area reduction compared to standard implementations on TSMC 7nm ASIC technology. However, these benefits largely rely on specialized hardware support, so meaningful acceleration is achieved mainly on FPGA or ASIC implementations rather than general-purpose processing units (e.g., CPU, GPU).

Koca et al. (2023) propose a training-free approximation technique to address the challenges of parallel processing difficulties and high hardware costs associated with softmax. This method changes the exponential base to 2 and separates it into integer and fractional parts, where the integer part is computed via shift operations and the fractional part is approximated using a simple linear function $v/2 + 1$. The final division is also replaced with shift operations through power-of-2 rounding. FPGA implementation results demonstrate $15\times$ and $55\times$ reductions in look-up table (LUT) and register usage, respectively, along with $23\times$ latency reduction compared to standard softmax, while maintaining performance with less than 1% accuracy degradation on BERT models.

ConSmax (Liu et al., 2024b) introduces an algorithmic modification to softmax computation by replacing the max and summation values with learnable fixed parameters. This approach directly addresses the reduced parallelism typically caused by synchronization requirements for these operations. Consequently, it enables completely independent and parallel computation of each output during inference. As a result, while maintaining nearly identical PPL to standard softmax on GPT2 models, the design and synthesis targeting 6nm ASIC technology achieved $7.5\times$ and $13.75\times$ improvements in power and area efficiency, respectively, compared to conventional standard softmax ASIC implementations. However, ConSmax has the limitation that it was only validated on a single dataset with small models under 100M parameters.

Baharav et al. (2024) observe that the full computation of standard softmax functions used in LLMs becomes prohibitively expensive when input dimensions are extremely large (e.g., LM head), and that only the top few high-probability outputs are typically significant rather than the entire probability distribution. Based on this insight, they propose an algorithm that efficiently approximates the top-k softmax output values through adaptive sampling. Instead of computing all logits during softmax calculation, this approach concentrates computational resources on estimating promising top candidate logits, similar to multi-armed bandits (Slivkins et al., 2019), thereby approximating the high-probability values of the entire softmax distribution at reduced cost. Experimental results demonstrate up to 32.6% FLOPs reduction when applied to the Mistral-7B model on WikiText.

LoLCATs (Zhang et al., 2025b) aims at linearizing pretrained LLMs by replacing quadratic softmax attention with sub-quadratic linear-attention analogs while avoiding expensive full-model retraining. It follows a two-stage pipeline: first, it performs attention transfer by learning layer-wise linear attention feature maps that regress to the original softmax attention outputs with an MSE loss; second, it swaps these analogs into the model and applies lightweight LoRA (Hu et al., 2022) only on the attention projections to recover language modeling quality. On LLaMA3 models from 8B up to larger variants, this closes roughly 80–87% of the quality gap between linearized and full softmax Transformers. However, a noticeable performance gap to the original softmax-attention models still remains on the most challenging benchmarks and long-context retrieval tasks.

Overall, these works illustrate that non-linear approximation in LLMs largely progresses along two intertwined fronts: softmax-centric accelerations and broader attention linearization. Hardware-oriented methods such as Softmax (Stevens et al., 2021) and the base-2 softmax of Koca et al. (2023) show that modest algorithmic changes—replacing exp with base-2 exponentials, max and division with shifts, and normalization with online or power-of-two schemes—can deliver multi $\times$ gains in energy, area, and latency on FPGA/ASIC implementations, often with sub-1% accuracy loss on BERT-scale models, but their benefits largely depend on specialized hardware support. ConSmax (Liu et al., 2024b) pushes this direction further by removing max/sum synchronization via learnable constants, enabling fully parallel output computation and achieving over $7\times$ and $13\times$ improvements in power and area efficiency on small GPT2-like models, though it remains validated on a narrow set of tasks and model scales. In contrast, adaptive top-k softmax (Baharav et al., 2024) targets the algorithmic cost of very large vocabularies by

focusing computation on high-probability logits, yielding up to 32.6% FLOPs reduction on Mistral-7B without modifying the underlying architecture. Finally, LoLCATs (Zhang et al., 2025b) indicates that replacing quadratic softmax attention with learned linear-attention analogs, combined with lightweight LoRA finetuning, can recover roughly 80–87% of the quality gap between linearized and full softmax Transformers on LLaMA-3 models, yet still leaves noticeable degradation on the hardest benchmarks and long-context retrieval. Taken together, these results suggest that non-linear approximation is most effective when it is co-designed with the target deployment substrate (ASIC/FPGA vs. general-purpose GPUs) and task regime (moderate vs. extreme sequence lengths), and that fully closing the quality gap for linearized attention in strong, long-context LLMs remains an open challenge.

4.5. Parameter sharing

Parameter sharing is one of the key techniques for reducing model size in LLMs. By sharing weights across different layers or components of the network, it significantly reduces the number of parameters while maintaining model performance. This technique is particularly actively explored in sub-billion parameter models, where constraints on model size and computational resources are more stringent.

A representative example is ALBERT (Lan et al., 2019), which reduces parameter size by factorizing the input embedding layer into two projection steps and shares parameters across them. In addition, ALBERT applies parameter sharing in the MHA, FFN, and LayerNorm modules across all Transformer layers. This design reduces the number of parameters from 108M in BERT-base to just 12M in ALBERT-base, and compresses ALBERT-large to 223M. Despite the aggressive compression, ALBERT-base achieves performance comparable to BERT-base on the GLUE benchmark, while ALBERT-xxlarge sets new state-of-the-art results on both GLUE and SQuADv2 at the time of its release.

Subformer (Reid et al., 2021) addresses the performance degradation caused by naive layer-wise parameter sharing by introducing a sandwich-style parameter sharing scheme, where only the first and last layers retain unique parameters, and the intermediate layers share the same parameters. This structure enables efficient architectures while mitigating performance losses from sharing.

Basis Sharing (Wang et al., 2024) proposes a method where each layer's weight matrix is decomposed via singular value decomposition (SVD), and reconstructed as a linear combination of shared basis vectors and layer-specific coefficients. This approach achieves more than 80% parameter reduction in LLaMA-7B with a PPL increase of just 0.6 on WikiText-2, and outperforms previous SVD-based methods (e.g., ASVD) reducing by up to 25% in PPL. When applied to OPT-6.7B and Mistral-7B, it yields 30% and 50% parameter reduction, respectively, with negligible performance degradation. Notably, in the case of Mistral-7B on GSM8K, it even improves accuracy by 4% compared to the baseline.

MobileLLM (Liu et al., 2024d) overcomes the inefficiency of existing studies that failed to systematically optimize parameter allocation in sub-billion models. The design first divides the repeated transformer layers into multiple groups and introduces a parameter sharing structure where layers within each group share the same parameters. In addition, parameter sharing is also applied between the input embedding and the linear layer used for output-logits generation. As a result, MobileLLM-350M achieves up to a 4.3% accuracy improvement compared to existing models of the same 350M scale, while MobileLLM-LS, which incorporates block-wise layer sharing, records an additional 0.8% accuracy gain.

MobiLLaMA (Thawakar et al., 2024) takes an even more aggressive approach by sharing the FFN module parameters across all Transformer layers. Given that the FFN typically accounts for around 65% of total parameters in standard Transformer architectures, this design achieves a substantial parameter reduction of up to 60%. Despite the compression, MobiLLaMA-0.5B surpasses the previous best-performing 0.5B-scale small language model (SLM) by an average of 2.4 percentage

points across nine benchmarks—including HellaSwag, ARC, PIQA, WinoGrande, OpenBookQA, BoolQ, RACE, CommonsenseQA, and SocialIQA—demonstrating the effectiveness of global parameter sharing in small-scale LLMs.

Relaxed Recursive Transformers (Bae et al., 2025) highlight the limitation of existing studies, where parameter sharing only scales with multiples of layer depth, and identical weights are forced to serve different roles at varying depths, resulting in performance degradation. This work converts a pretrained LLM into a Recursive Transformer by repeatedly applying a single block, while attaching layer-wise LoRA modules at each recurrence to relax weight tying. With only a small amount of uptraining, this approach preserves accuracy close to the original model and improves theoretical inference throughput. However, the method introduces a serving limitation in batched inference, since each block requires a separate LoRA module computation, reducing parallelism. While concatenating LoRA weights can alleviate this issue, it also causes redundant computations. Therefore, practical deployment would require custom CUDA kernels for LoRA serving and inter-accelerator parallelization.

Overall, the reported quantitative results provide concrete evidence that parameter sharing involves balancing three axes: the location of sharing, the aggressiveness of sharing, and the amount of layer-specific freedom retained. ALBERT-base (Lan et al., 2019) compresses BERT-base from 108M to 12M parameters while maintaining similar GLUE performance, Basis Sharing (Wang et al., 2024) reduces parameters of LLaMA-7B by more than 80% with only a 0.6 increase in WikiText-2 perplexity and even improves GSM8K accuracy of Mistral-7B by 4 percentage points at 50% parameter reduction, and MobileLLM-350M (Liu et al., 2024d) together with MobiLLaMA-0.5B (Thawakar et al., 2024) reports up to roughly 4–5 percentage-point accuracy gains while reducing parameters by up to 60% through group-wise or FFN sharing. These results collectively indicate that globally or group-wise sharing of highly redundant modules (e.g., FFNs or low-rank bases), combined with a small amount of layer-specific capacity (e.g., coefficients or lightweight adapters), is particularly effective for simultaneously achieving efficiency and accuracy in sub-billion-parameter LLMs. In contrast, Relaxed Recursive Transformers (Bae et al., 2025) reveal that repeatedly applying a single shared block, while theoretically increasing compute reuse and throughput and preserving accuracy close to the original model with only modest uptraining, can introduce practical serving bottlenecks: each recurrence requires a separate LoRA module computation, which reduces parallelism in batched inference, and merging LoRA weights to mitigate this issue leads to redundant computations. Overall, these quantitative findings suggest that parameter sharing should not be viewed as a binary choice between fully tied and fully independent parameters. Instead, it functions as a key design axis that jointly optimizes sharing ratios, per-layer freedom, and serving structure, thereby determining the operating point between efficiency in sub-billion-parameter LLMs and practical deployment performance in larger models.

5. Discussion

5.1. Cross-technology comparison

This section compares different technique categories (i.e., quantization, pruning, and KD) along common quantitative axes: parameter compression, FLOPs reduction, memory saving, and accuracy degradation. Such a comparison highlights the trade-offs among memory, computation, and accuracy, provides a quantitative basis for deriving practical design choices for efficient models, and offers clues to potential complementarities across techniques (e.g., combining quantization with pruning, or KD with static compression). The values in the table are primarily taken from the original papers, while missing entries are supplemented by our own measurements on checkpoints released on Hugging Face.

Table 4 summarizes cross-technology results on LLaMA-family models around 7–8B parameters and reports, for each method, accuracy

Table 4

Cross-technology comparison of LLM compression techniques (quantization, pruning, and KD) on LLaMA series in terms of memory, speedup, and accuracy trade-offs.

Categories	Techniques	Model	Compression	Acc. degrad.	FLOPs reduction	Memory saving	Speedup / Hardware
Quantization	AQLM (Egiazarian et al., 2024)	LLaMA2-7B	W3/A16	1.47%	–	72.86%	1.20 / 1 RTX3090 GPU
	LeanQuant (Zhang & Shrivastava, 2025)	LLaMA3-8B	W4/A16	1.12%	–	50.09%	– / –
	QLLM (Liu et al., 2023)	LLaMA-7B	W4/A16	1.39%	–	–	1.96 / 1 RTX3090 GPU
	SpinQuant (Liu et al., 2025b)	LLaMA3-8B	W4/A4	3.8%	–	–	2.78 / 1 M1 Pro-CPU
	OSTQuant (Hu et al., 2025b)	LLaMA2-7B	W4/A4	2.95%	–	62.85%	1.89 / 1 A6000 GPU
	EfficientQAT (Chen et al., 2025)	LLaMA2-7B	W3/A16	0.84%	–	75.98%	2.92 / 1 A100 GPU
Pruning	SliceGPT (Ashkboos et al., 2024)	LLaMA2-7B	30% ⁽¹⁾	15.30%	47.31%	31.00%	1.36 / 1 H100 GPU
	Wanda (Sun et al., 2024a)	LLaMA2-7B	50% ⁽²⁾	4.28%	59.00%	41.21%	1.24 / 2 A6000 GPU
	LLM-Pruner (Ma et al., 2023)	LLaMA-7B	20% ⁽³⁾	5.02%	19.90%	19.47%	1.16 / 1 A5000 GPU
	SlimLLM (Guo et al., 2025)	LLaMA-7B	50% ⁽³⁾	17.32%	46.90%	49.30%	1.30 / 1 V100 GPU
	PROBE PRUNING (Le et al., 2025)	LLaMA2-7B	20% ⁽⁴⁾	1.88%	32.69%	–	1.14 / 1 A100 GPU
			40% ⁽⁴⁾	11.56%	60.32%	–	1.30 / 1 A100 GPU
KD	TSLD (Kim et al., 2023)	LLaMA-7B	W3/A16	4.12%	–	87.50%	1.27 / 1 A100 GPU
	DSKD (Zhang et al., 2024)	LLaMA-7B	TinyLlama-1.1B	-0.57%	34.59%	83.68%	1.58 / 1 H100 GPU
	BitDistiller (Du et al., 2024)	LLaMA3.1-8B	W2/A16	6.03%	–	87.50%	1.27 / 1 H100 GPU
			W3/A16	37.11%	–	81.25%	1.14 / 1 H100 GPU
	Proxy-KD (Chen et al., 2024)	LLaMA2-7B	70B(Proxy)	36.19%	79.00%	79.00%	13 / 4 H100 GPU

Acc. degrad.: positive = drop, negative = improvement

(1) SliceGPT: fraction of total model parameters removed, including embeddings.

(2) Wanda: uniform unstructured sparsity on all linear layers; embedding and LM head remain dense (~1% of params).

(3) LLM-Pruner and SlimLLM: approximate fraction of total model parameters removed by structured pruning in attention and FFN.

(4) PROBE PRUNING: average weight pruning ratio over attention and FFN blocks in layers 4–32; layers 1–3 are kept dense.

degradation, FLOPs reduction, memory saving, and end-to-end speedup. For quantization and pruning, we mainly use WikiText-2 metrics when they are available. For KD, TSLD (Kim et al., 2023) is finetuned on multiple task datasets (*i.e.*, PIQA, OpenBookQA, ARC, and GSM8K), DSKD (Zhang et al., 2024) is evaluated on instruction-following benchmarks (*i.e.*, Dolly, SelfInstruct, VicunaEval, S-NI, and UnNI) using Rouge-L, BitDistiller (Du et al., 2024) uses WikiText-2, and Proxy-KD (Chen et al., 2024) distills from GPT-4 (Achiam et al., 2023) style responses such as OpenOrca (Open-Orca Contributors, 2023) and Nectar (Zhu et al., 2024). Speedups are measured as the ratio of tokens/s before and after compression, with the measurement hardware specified in the table.

Viewed along the accuracy–memory plane, among methods that report memory usage, 3–4 bit quantization of LLaMA2/3 7–8B models yields roughly 50–76% memory savings with at most 3.8% accuracy degradation: LeanQuant (Zhang & Shrivastava, 2025) attains 50.09% memory saving with a 1.12% accuracy drop on LLaMA3-8B, AQLM (Egiazarian et al., 2024) and EfficientQAT (Chen et al., 2025) achieve 72.86% and 75.98% savings with 1.47% and 0.84% drops on LLaMA2-7B, and OSTQuant (Hu et al., 2025b) reports 62.85% saving with 2.95% degradation. Pruning methods that remove a comparable or larger fraction of parameters save less memory for a given accuracy loss: Wanda (Sun et al., 2024a) reaches 41.21% memory saving at 50% sparsity with a 4.28% drop, SlimLLM (Guo et al., 2025) obtains 49.30% saving with 17.32% drop at 50% pruning, SliceGPT (Ashkboos et al., 2024) saves 31.00% memory with a 15.30% drop at 30% parameter removal, and LLM-Pruner (Ma et al., 2023) saves only 19.47% memory at 5.02% degradation for 20% structured pruning. KD-based methods extend this trade-off toward much higher compression: DSKD (Zhang et al., 2024) compresses LLaMA-7B into TinyLlama-1.1B with 83.68% memory saving and a small accuracy improvement reported as -0.57% degradation, TSLD (Kim et al., 2023) reaches 87.50% saving with a 4.12% drop, and BitDistiller (Du et al., 2024) in its 2-bit configuration achieves 87.50% saving with 6.03% drop. Compared with quantization and pruning, KD therefore occupies a high-compression region of the accuracy–memory plane around 80–88% memory saving and spans outcomes from accuracy gains, as in DSKD, to moderate losses, as in TSLD and BitDistiller, whereas pruning in our table rarely exceeds 50% memory reduction even at aggressive sparsity levels.

Along the FLOPs–speed axis, pruning and KD explicitly reduce arithmetic operations, whereas quantization mainly improves constant factors in dense computation. Wanda (Sun et al., 2024a), SliceGPT (Ashkboos et al., 2024), SlimLLM (Guo et al., 2025), LLM-Pruner (Ma et al., 2023), and PROBE PRUNING (Le et al., 2025) report FLOPs reductions between 19.90% and 60.32%, yet their measured speedups span only 1.14–1.36×, which reveals a clear gap between theoretical sparsity and realized throughput on current GPU kernels. For example, Wanda reduces FLOPs by 59.00% but accelerates inference by only 1.24×, and PROBE PRUNING at 40% sparsity reduces FLOPs by 60.32% with a 1.30× speedup. Quantization, in contrast, leaves FLOPs unchanged but attains comparable or larger speedups purely from denser low-precision math and lower memory traffic: QLLM (Liu et al., 2023) achieves a 1.96× speedup on an RTX 3090 GPU, EfficientQAT (Chen et al., 2025) reaches 2.92× on an A100 GPU, SpinQuant (Liu et al., 2025b) reports 2.78× on an M1 Pro-CPU, OSTQuant (Hu et al., 2025b) delivers 1.89× on an A6000 GPU, and AQLM (Egiazarian et al., 2024) on a single RTX 3090 still yields a 1.20× speedup. KD-based methods lie between these two behaviors: DSKD (Zhang et al., 2024) reduces FLOPs by 34.59% and achieves a 1.58× speedup with an architecturally smaller TinyLlama-1.1B student, while Proxy-KD (Chen et al., 2024) goes much further by reporting 79.00% FLOPs reduction and a 13× speedup on 4 H100 GPUs through distillation from a 70B teacher to a 7B student. These numbers indicate that for moderate speed targets of roughly 2–3×, low-bit quantization alone often converts compression into throughput more effectively than sparsity-based pruning, whereas KD enables the most aggressive FLOPs and latency reductions when very small students are acceptable.

Taken together, these axis-wise comparisons show that quantization, pruning, and KD are not interchangeable knobs but occupy distinct regions of the global accuracy–efficiency space. Quantization clusters around moderate accuracy degradation with strong memory savings and speedups up to about 3×, pruning shifts the balance toward larger FLOPs reductions but delivers weaker memory savings and more fragile accuracy at similar compression levels, and KD stretches the space toward both accuracy-improving small students, as in DSKD (Zhang et al., 2024), and extremely compressed but less accurate students, as in Proxy-KD (Chen et al., 2024) and high-compression Bit-

Distiller (Du et al., 2024). Overall, the strongest operating points observed in Table 4 already arise from combinations that cross technique families, for example using KD together with low-bit quantization in TSLD (Kim et al., 2023) and BitDistiller (Du et al., 2024), or adding activation-based pruning, as in PROBE PRUNING (Le et al., 2025), on top of static compression such as quantization or weight pruning; this suggests that hybrid cross-technology designs provide an effective means to extend the achievable trade-off region.

5.2. Robustness and sensitivity of compression methods

Model compression methods should be evaluated not only by the degree of compression they achieve, but also by their sensitivity to hyperparameter choices and shifts in data distribution. Representative techniques such as quantization, pruning, and KD are each affected by different design variables. The cross-technology comparison in Table 4 further highlights the distinct trade-offs these methods present in terms of accuracy degradation, memory savings, and computational reduction. This section summarizes the representative sensitivity trends reported for each compression method.

5.2.1. Quantization

For quantization methods, the main sensitivity factors are bit-width and calibration strategy. Lower bit-widths generally provide higher compression ratios, but at the cost of larger accuracy degradation. For example, GPTQ (Frantar et al., 2022) reported WikiText-2 perplexities of 8.34, 8.37, and 8.68 for FP16, 4-bit, and 3-bit quantization of OPT-175B, respectively, indicating that 4-bit quantization causes negligible accuracy loss, whereas 3-bit quantization leads to a more noticeable degradation. AWQ (Lin et al., 2024), which adopts activation-aware scaling, further showed that protecting salient weight channels based on activation statistics can effectively reduce quantization error. In particular, AWQ reported a WikiText-2 perplexity of 3.95 for 3-bit weight-only quantization of LLaMA-65B, suggesting that performance can be preserved even under low-bit quantization. The results in Table 4 further support these trends, showing that 3–4 bit quantization of LLaMA-family models provides roughly 50–76% memory savings while limiting accuracy degradation to within 3.8%.

5.2.2. Pruning

For pruning methods, the primary sensitivity factors are the sparsity ratio and pruning criterion. For example, PROBE PRUNING (Le et al., 2025) showed that increasing the pruning ratio from 20% to 40% on LLaMA-2-7B reduced the average zero-shot accuracy from 68.5 to 61.7, and increased the WikiText-2 perplexity from 8.1 to 16.8, highlighting the sensitivity of model performance to the pruning ratio. Additionally, Wanda (Sun et al., 2024a) reported that the average zero-shot accuracy of LLaMA-7B decreased from 59.99 to 54.21 under 50% unstructured sparsity, whereas SlimLLM (Guo et al., 2025) showed that the average accuracy of LLaMA2-7B dropped from 62.89 to 46.93 under 50% structured pruning. These findings indicate that, beyond the sparsity ratio, the pruning criterion and structural design choices also play important roles in determining model performance.

5.2.3. Knowledge distillation

Unlike quantization or pruning, KD is not governed by a single monotonic trade-off variable such as bit-width or sparsity ratio. Instead, student performance depends strongly on data and configuration choices, including the teacher model, the composition of the distillation data, and the combination of training objectives. For example, DSKD (Zhang et al., 2024) reported that using diverse instruction-tuning datasets for distillation improves the instruction-following performance of student models. Furthermore, Proxy-KD (Chen et al., 2024) showed that a larger teacher model substantially improves student performance, whereas reducing teacher scale makes it more difficult to preserve performance.

5.2.4. Approximation

Approximation-based methods replace attention or softmax operations with approximate functions and are primarily sensitive to the approximation scheme and the target hardware environment. In particular, several studies (Choromanski et al., 2020; Wang et al., 2020) have reported that certain approximation methods provide greater performance benefits under specific hardware settings.

5.2.5. Parameter sharing

For parameter-sharing methods, the selection of shared layers and the degree of sharing are key design variables. For example, ALBERT (Lan et al., 2019) demonstrated that cross-layer parameter sharing can substantially reduce model size while maintaining performance. Subsequent work (Wang et al., 2024) proposed combining shared basis vectors with layer-specific coefficients to compensate for the representational capacity lost through sharing.

5.3. Design guidelines across deployment scenarios

This paper investigates module structure optimization and compression techniques for transformer-based LLMs, including the recent research. Building on this foundation, this section moves beyond isolated approaches to present integrated design guidelines tailored to specific deployment scenarios, combining multiple techniques to optimize both efficiency and performance.

Long-context scenario. When processing input sequences exceeding 16k–32k tokens, models face both accuracy degradation and increased computational cost due to the extended context length. To alleviate these challenges, the positional encoding can be replaced with extrapolation-robust methods such as CoPE (Golovneva et al., 2024) or YaRN (Peng et al., 2024), which better preserve representational capacity across long contexts. In parallel, the attention module can be substituted with architectures like Performer (Choromanski et al., 2020), which provide linear or sub-quadratic time complexity with respect to sequence length, thereby reducing the computational burden. Crucially, this linearization can be strategically paired with activation pruning such as PROBE PRUNING to exploit input-dependent sparsity in the now-linearized attention computations, enabling pruning to cover a broader fraction of operations and thus yielding higher effective sparsity. Indeed, the Effectiveness of this synergy has been recently demonstrated by SLA (Zhang et al., 2025a). By combining linear attention and sparsity in transformer-based models, SLA reduced attention computations by 95% and accelerated overall inference speed by 2.2× without quality degradation. However, such linearization and sparsification approaches can degrade precise information-matching capabilities, potentially leading to accuracy drops in Needle-in-a-Haystack (NIAH) tasks that require extracting specific proper nouns.

Fast-response scenario. For latency-sensitive applications (e.g., assistants or real-time search) where per-token latency must stay within 10–50 ms, a practical guideline is to first secure an accurate yet compact backbone via Proxy-KD to obtain a strong LLaMA2-7B baseline (Chen et al., 2024). On top of this, applying structured pruning such as LLM-Pruner (Ma et al., 2023)—which removes FFN channels and attention heads using block-sparse patterns—can yield about 10–20% FLOP reductions and speed up token generation without runtime branching overheads. Complementarily, adopting Luna (Ma et al., 2021) to replace softmax-based attention with a linear formulation helps eliminate a major latency bottleneck. With attention thus linearized, activation-sparsity methods such as ProSparse (Song et al., 2025) can be layered in to exploit input-dependent sparsity, enabling additional acceleration while keeping small accuracy degradation. The effectiveness of this ensemble is well-supported by recent literature. For instance, SPQ (Yao & Gultepe, 2026) successfully harmonizes attention SVD, FFN pruning, and 8-bit

quantization to achieve a 1.9× speedup. Unlike SPQ, which primarily relies on PTQ and low-rank approximations, our proposed guideline utilizes Luna (Ma et al., 2021) to linearize attention and fundamentally expands the operational space for activation-sparsity methods like ProSparse (Song et al., 2025), enabling more aggressive latency reductions suitable for fast-response scenarios. Nevertheless, because these methods involve non-linear function approximation and block-wise pruning, context comprehension may decline, potentially causing performance degradation in coding or precise parsing tasks.

Edge-device scenario. To overcome the severely constrained memory bandwidth and capacity of edge-device environments, this guideline proposes a multi-faceted optimization pipeline across the model. First, GQA (Ainslie et al., 2023) is introduced to reduce dynamic KV cache traffic. Subsequently, parameter sharing techniques like MobilLaMA (Thawakar et al., 2024) eliminate static weight redundancy across FFNs, which, when paired with structural pruning like SliceGPT, creates a synergy that simultaneously reduces all FFN parameters. To recover performance drops accumulated during compression, reverse KL-based KD like MiniLLM (Gu et al., 2024) is applied, efficiently transferring logit distributions without complex intermediate layer mappings. Finally, methods like EfficientQAT (Chen et al., 2025) can be utilized for light weighting while minimizing accuracy loss. The efficacy of such composite compression methods is validated by recent research (Chhawri et al., 2025), and our guideline is distinctly differentiated by systematically combining parameter sharing and pruning. However, since this pipeline heavily prunes and linearizes the model, the general representational capacity may decrease, causing a significant drop in response quality for few-shot or complex mathematical reasoning tasks.

High-accuracy scenario. When accuracy is paramount and hardware resources are not tightly constrained, it is reasonable to prioritize techniques that enhance the model's expressivity. For stabilizing deep residual paths, a Pre-LN backbone (Xiong et al., 2020) with RMSNorm (Zhang & Sennrich, 2019) often proves effective; Mix-LN (Li et al., 2024) in the initial shallow layers can help amplify gradient signals, with a transition to Pre-LN in deeper layers. For positional encoding, RoPE (Su et al., 2024) serves as a strong default, while context-extension mechanisms such as YaRN (Peng et al., 2024) or CoPE (Golovneva et al., 2024) tend to improve long-context generalization. GLU-style FFNs are frequently effective for accuracy (Shazeer, 2020); and when additional expressivity is desired, an MoE approach—such as MoEfication (Zhang et al., 2022b) or DeepSeekMoE (Dai et al., 2024)—can be adopted to improve accuracy while keeping per-token compute bounded. Existing studies covering models like Llama (Touvron et al., 2023b), GPT (Frantar et al., 2022), Qwen (Bai et al., 2023a), and Mixtral (Jiang et al., 2024) typically treat quantization as a post-hoc optimization step. In contrast, our guideline recommends allocating the hardware resources saved by quantization into deeper layer stacking and wider MoE architectures, proposing a design that maximizes expressivity and capacity within the same hardware budget. However, stacking deeper layers carries the risk that minute quantization errors or activation outliers from the initial shallow layers will propagate and accumulate in the deeper layers, potentially degrading performance in multi-step reasoning or long-form generation tasks.

6. Conclusions

This paper provides a comprehensive survey of model-level optimization techniques for efficient LLM inference. We review recent advances in efficient module designs for core transformer components—MHA, FFN, positional encoding, and normalization—as well as representative compression methods such as quantization, pruning, knowledge distillation, approximation, and parameter sharing. Building on this foundation, we proposed design strategies that integrate complementary techniques tailored to the specific tasks of long-context process-

ing, low-latency inference, edge-device deployment, and high-accuracy requirements. We believe that the practical guidelines proposed in this study will serve as valuable references for developing LLMs optimized for deployment in diverse industrial scenarios.

CRediT authorship contribution statement

Gilhyeon Lee: Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Seonggeun Kim:** Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Dongjun Lee:** Writing – original draft, Methodology, Investigation, Formal analysis, Data curation; **Kyungmin Goh:** Writing – original draft, Methodology, Investigation, Formal analysis, Data curation; **Hyun Kim:** Writing – review & editing, Supervision, Project administration, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was supported by K-CHIPS(Korea Collaborative & High-tech Initiative for Prospective Semiconductor Research)(2410000620, RS-2024-00405946, 24052-15TC) funded by the Ministry of Trade, Industry & Energy (MOTIE, Korea) and the MSIT(Ministry of Science and ICT), Korea, under the ITRC(Information Technology Research Center) support program (IITP-2026-RS-2022-00156295) supervised by the IITP(Institute for Information & Communications Technology Planning & Evaluation).

References

- Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altschmidt, J., Altman, S., Anadkat, S. et al. (2023). Gpt-4 technical report. *arXiv:2303.08774*.
- Adlakha, V., Dhuliawala, S., Suleman, K., de Vries, H., & Reddy, S. (2022). Topiocqa: Open-domain conversational question answering with topic switching. *Transactions of the Association for Computational Linguistics*, 10, 468–483.
- Agarwal, R., Vieillard, N., Stanczyk, P., Ramos, S., Geist, M., & Bachem, O. (2023). Gkd: Generalized knowledge distillation for auto-regressive sequence models. 12, *arXiv:2306.1349*.
- Ainslie, J., Lee-Thorp, J., De Jong, M., Zemlyanskiy, Y., Lebrón, F., & Sanghai, S. (2023). Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv:2305.13245*.
- Almazrouei, E., Alobeidli, H., Alshamsi, A., Cappelli, A., Cojocaru, R., Debbah, M., Goffinet, É., Hessel, D., Launay, J., Malartic, Q. et al. (2023). The falcon series of open language models. *arXiv:2311.16867*.
- Anantha, R., Vakulenko, S., Tu, Z., Longpre, S., Pulman, S., & Chappidi, S. (2020). Open-domain question answering goes conversational via question rewriting. *arXiv:2010.04898*.
- Ashkboos, S., Croci, M. L., Nascimento, M. G. d., Hoefler, T., & Hensman, J. (2024). Slicept: Compress large language models by deleting rows and columns. *arXiv:2401.15024*.
- Ba, J. L., Kiros, J. R., & Hinton, G. E. (2016a). Layer normalization. *arXiv:1607.06450*.
- Ba, J. L., Kiros, J. R., & Hinton, G. E. (2016b). Layer normalization. *Stat*, 1050, 21.
- Bachlechner, T., Majumder, B. P., Mao, H., Cottrell, G., & McAuley, J. (2021). Rezero is all you need: Fast convergence at large depth. In *Uncertainty in artificial intelligence* (pp. 1352–1361). PMLR.
- Bae, S., Fisch, A., Harutyunyan, H., Ji, Z., Kim, S., & Schuster, T. (2025). Relaxed recursive transformers: Effective parameter sharing with layer-wise loRA. In *The thirteenth international conference on learning representations*. <https://openreview.net/forum?id=WwpYSOkkCt>.
- Baharav, T., Kang, R., Sullivan, C., Tiwari, M., Luxenberg, E., Tse, D., & Pilanci, M. (2024). Adaptive sampling for efficient softmax approximation. *Advances in Neural Information Processing Systems*, 37, 117580–117613.
- Bai, G., Chai, Z., Ling, C., Wang, S., Lu, J., Zhang, N., Shi, T., Yu, Z., Zhu, M., Zhang, Y. et al. (2024a). Beyond efficiency: A systematic survey of resource-efficient large language models. *arXiv:2401.00625*.
- Bai, G., Li, Y., Ling, C., Kim, K., & Zhao, L. (2024b). SparseLLM: Towards global pruning of pre-trained language models. *Advances in Neural Information Processing Systems*, 37, 46203–46225.

- Bai, J., Bai, S., Chu, Y., Cui, Z., Dang, K., Deng, X., Fan, Y., Ge, W., Han, Y., Huang, F. et al. (2023a). Qwen technical report. [arXiv:2309.16609](https://arxiv.org/abs/2309.16609).
- Bai, Y., Lv, X., Zhang, J., Lyu, H., Tang, J., Huang, Z., Du, Z., Liu, X., Zeng, A., Hou, L. et al. (2023b). Longbench: A bilingual, multitask benchmark for long context understanding. [arXiv:2308.14508](https://arxiv.org/abs/2308.14508).
- Bajaj, P., Campos, D., Craswell, N., Deng, L., Gao, J., Liu, X., Majumder, R., McNamara, A., Mitra, B., Nguyen, T. et al. (2016). Ms marco: A human generated machine reading comprehension dataset. [arXiv:1611.09268](https://arxiv.org/abs/1611.09268).
- Banerjee, P., Pal, K. K., Mitra, A., & Baral, C. (2019). Careful selection of knowledge to solve open book question answering. [arXiv:1907.10738](https://arxiv.org/abs/1907.10738).
- Banerjee, S., & Lavie, A. (2005). Meteor: An automatic metric for mt evaluation with improved correlation with human judgments. In *Proceedings of the acl workshop on intrinsic and extrinsic evaluation measures for machine translation and/or summarization* (pp. 65–72).
- Barros, S. (2025). Solving AI foundational model latency with telco infrastructure. [arXiv:2504.03708](https://arxiv.org/abs/2504.03708).
- Beck, M., Pöppel, K., Lippe, P., & Hochreiter, S. (2025). Tiled flash linear attention: More efficient linear RNN and xLSTM kernels. [arXiv:2503.14376](https://arxiv.org/abs/2503.14376).
- Beltagy, I., Peters, M. E., & Cohan, A. (2020). Longformer: The long-document transformer. [arXiv:2004.05150](https://arxiv.org/abs/2004.05150).
- Bisk, Y., Zellers, R., Gao, J., Choi, Y. et al. (2020). Piqa: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 7432–7439). (vol. 34).
- Bojar, O., Buck, C., Federmann, C., Haddow, B., Koehn, P., Leveling, J., Monz, C., Pecina, P., Post, M., Saint-Amand, H. et al. (2014). Findings of the 2014 workshop on statistical machine translation. In *Proceedings of the ninth workshop on statistical machine translation* (pp. 12–58).
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Nee-lakantan, A., Shyam, P., Sastry, G., Askell, A. et al. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901.
- Chelba, C., Mikolov, T., Schuster, M., Ge, Q., Brants, T., Koehn, P., & Robinson, T. (2013). One billion word benchmark for measuring progress in statistical language modeling. [arXiv:1312.3005](https://arxiv.org/abs/1312.3005).
- Chen, H., Chen, R., Yi, Y., Quan, X., Li, C., Yan, M., & Zhang, J. (2024). Knowledge distillation of black-box large language models. [arXiv:2401.07013](https://arxiv.org/abs/2401.07013).
- Chen, M., Shao, W., Xu, P., Wang, J., Gao, P., Zhang, K., & Luo, P. (2025). Efficient quantization-aware training for large language models. In W. Che, J. Nabende, E. Shutova, & M. T. Pilehvar (Eds.), *Proceedings of the 63rd annual meeting of the association for computational linguistics (volume 1: Long papers)* (pp. 10081–10100). Vienna, Austria: Association for Computational Linguistics. <https://doi.org/10.18653/v1/2025.acl-long.498>.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., Oliveira, P. H. P. D., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G. et al. (2021). Evaluating large language models trained on code. [arXiv:2107.03374](https://arxiv.org/abs/2107.03374).
- Chen, S., Wong, S., Chen, L., & Tian, Y. (2023). Extending context window of large language models via positional interpolation. [arXiv:2306.15595](https://arxiv.org/abs/2306.15595).
- Cheng, H., Zhang, M., & Shi, J. Q. (2024). A survey on deep neural network pruning: Taxonomy, comparison, analysis, and recommendations. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(12), 10558–10578. <https://doi.org/10.1109/TPAMI.2024.3447085>.
- Chhawri, S., Mahadik, R., & Roj, S. (2025). A systematic study of compression ordering for large language models. [arXiv:2511.19495](https://arxiv.org/abs/2511.19495).
- Choromanski, K., Likhoshershtov, V., Dohan, D., Song, X., Kane, A., Sarlos, T., Hawkins, P., Davis, J., Mohiuddin, A., Kaiser, L. et al. (2020). Rethinking attention with performers. [arXiv:2009.14794](https://arxiv.org/abs/2009.14794).
- Chowdhery, A., Narang, S., Devlin, J., Bosma, M., Mishra, G., Roberts, A., Barham, P., Chung, H. W., Sutton, C., Gehrmann, S. et al. (2023). Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24(240), 1–113.
- Chua, V. S., Pan, Y., & Jain, N. (2024). Post-training statistical calibration for higher activation sparsity. [arXiv:2412.07174](https://arxiv.org/abs/2412.07174).
- Clark, C., Lee, K., Chang, M.-W., Kwiatkowski, T., Collins, M., & Toutanova, K. (2019). Boolq: Exploring the surprising difficulty of natural yes/no questions. [arXiv:1905.10044](https://arxiv.org/abs/1905.10044).
- Clark, P., Cowhey, I., Etzioni, O., Khot, T., Sabharwal, A., Schoenick, C., & Tafjord, O. (2018). Think you have solved question answering? Try arc, the ai2 reasoning challenge. [arXiv:1803.05457](https://arxiv.org/abs/1803.05457).
- Clement, C. B., Bierbaum, M., O’Keeffe, K. P., & Alemi, A. A. (2019). On the use of arxiv as a dataset. [arXiv:1905.00075](https://arxiv.org/abs/1905.00075).
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R. et al. (2021). Training verifiers to solve math word problems. [arXiv:2110.14168](https://arxiv.org/abs/2110.14168).
- Dai, D., Deng, C., Zhao, C., Xu, R. x., Gao, H., Chen, D., Li, J., Zeng, W., Yu, X., Wu, Y., Xie, Z., Li, Y. k., Huang, P., Luo, F., Ruan, C., Sui, Z., & Liang, W. (2024). DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models. In L.-W. Ku, A. Martins, & V. Srikumar (Eds.), *Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 1: Long papers)* (pp. 1280–1297). Bangkok, Thailand: Association for Computational Linguistics. <https://doi.org/10.18653/v1/2024.acl-long.70>.
- Dai, Z., Yang, Z., Yang, Y., Carbonell, J. G., Le, Q., & Salakhutdinov, R. (2019). Transformer-XL: Attentive language models beyond a fixed-length context. In *Proceedings of the 57th annual meeting of the association for computational linguistics* (pp. 2978–2988).
- Dao, T. (2024). Flashattention-2: Faster attention with better parallelism and work partitioning. In *The twelfth international conference on learning representations*. <https://openreview.net/forum?id=mZn2Xyh9Ec>.
- Dao, T., Fu, D., Ermon, S., Rudra, A., & Ré, C. (2022). Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems*, 35, 16344–16359.
- Dasgupta, S., & Cohn, T. (2025). Improving language model distillation through hidden state matching. In *The thirteenth international conference on learning representations*.
- Dernoncourt, F., & Lee, J. Y. (2017). Pubmed 200k rct: A dataset for sequential sentence classification in medical abstracts. [arXiv:1710.06071](https://arxiv.org/abs/1710.06071).
- Dettmers, T., Lewis, M., Belkada, Y., & Zettlemoyer, L. (2022). Gpt3. int8: 8-Bit matrix multiplication for transformers at scale. *Advances in Neural Information Processing Systems*, 35, 30318–30332.
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the north american chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)* (pp. 4171–4186).
- Dolan, W. B., & Brockett, C. (2005). Automatically constructing a corpus of sentential paraphrases. In *Proceedings of the third international workshop on paraphrasing (IWP2005)* (p. N/A). <https://aclanthology.org/105-5002/>.
- Dong, P., Li, L., Zhong, Y., Du, D., FAN, R., Chen, Y., Tang, Z., Wang, Q., Xue, W., Guo, Y., & Chu, X. (2025). STBLLM: Breaking the 1-bit barrier with structured binary LLMs. In *The thirteenth international conference on learning representations*. <https://openreview.net/forum?id=6XUSDvBFkV>.
- Du, D., Zhang, Y., Cao, S., Guo, J., Cao, T., Chu, X., & Xu, N. (2024). Bitdistiller: Unleashing the potential of sub-4-bit LLMs via self-distillation. In *Acl (1)* (pp. 102–116). <https://doi.org/10.18653/v1/2024.acl-long.7>.
- Du, N., Huang, Y., Dai, A. M., Tong, S., Lepikhin, D., Xu, Y., Krikun, M., Zhou, Y., Yu, A. W., Firat, O., et al. (2022). Glam: Efficient scaling of language models with mixture-of-experts. In *International conference on machine learning* (pp. 5547–5569). PMLR.
- Egiazarian, V., Panferov, A., Kuznedelev, D., Frantar, E., Babenko, A., & Alistarh, D. (2024). Extreme compression of large language models via additive quantization. In *Forty-first international conference on machine learning*. <https://openreview.net/forum?id=5mCaITRTmO>.
- Fabbri, A. R., Li, I., She, T., Li, S., & Radev, D. R. (2019). Multi-news: A large-scale multi-document summarization dataset and abstractive hierarchical model. [arXiv:1906.01749](https://arxiv.org/abs/1906.01749).
- Fedus, W., Zoph, B., & Shazeer, N. (2022). Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120), 1–39.
- Frantar, E., & Alistarh, D. (2022). Optimal brain compression: A framework for accurate post-training quantization and pruning. *Advances in Neural Information Processing Systems*, 35, 4475–4488.
- Frantar, E., & Alistarh, D. (2023). SparseGPT: Massive language models can be accurately pruned in one shot. In *Proceedings of the 40th international conference on machine learning - IJMLR*. <https://jmlr.org>.
- Frantar, E., Ashkboos, S., Hoefler, T., & Alistarh, D. (2022). Gptq: Accurate post-training quantization for generative pre-trained transformers. [arXiv:2210.17323](https://arxiv.org/abs/2210.17323).
- Gao, L., Biderman, S., Black, S., Golding, L., Hoppe, T., Foster, C., Phang, J., He, H., Thite, A., Nabeshima, N., Presser, S., & Leahy, C. (2020). The Pile: An 800GB dataset of diverse text for language modeling. [arXiv:2101.00027](https://arxiv.org/abs/2101.00027).
- GLM, T., Zeng, A., Xu, B., Wang, B., Zhang, C., Yin, D., Zhang, D., Rojas, D., Feng, G., Zhao, H. et al. (2024). Chatglm: A family of large language models from glm-130b to glm-4 all tools. [arXiv:2406.12793](https://arxiv.org/abs/2406.12793).
- Golovneva, O., Wang, T., Weston, J., & Sukhbaatar, S. (2024). Contextual position encoding: Learning to count what’s important. [arXiv:2405.18719](https://arxiv.org/abs/2405.18719).
- Golovneva, O., Wang, T., Weston, J., & Sukhbaatar, S. (2025). Multi-token attention. [arXiv:2504.00927](https://arxiv.org/abs/2504.00927).
- Gu, Y., Dong, L., Wei, F., & Huang, M. (2024). MiniLLM: Knowledge distillation of large language models. In *The twelfth international conference on learning representations*. <https://openreview.net/forum?id=5h0qf7IBZZ>.
- Guo, J., Chen, X., Tang, Y., & Wang, Y. (2025). SlimLLM: Accurate structured pruning for large language models. [arXiv:2505.22689](https://arxiv.org/abs/2505.22689).
- Gupta, A., Guo, H., Yuan, Y., Zhou, Y., & Mendis, C. (2024). Flurka: Fast and accurate unified low-rank & kernel attention. [arXiv:2306.15799](https://arxiv.org/abs/2306.15799).
- Hamman, F., Dissanayake, P., Fu, Y., & Dutta, S. (2025). Few-shot knowledge distillation of LLMs with counterfactual explanations. [arXiv:2510.21631](https://arxiv.org/abs/2510.21631).
- He, P., Liu, X., Gao, J., & Chen, W. (2020). DeBERTa: Decoding-enhanced bert with disentangled attention. [arXiv:2006.03654](https://arxiv.org/abs/2006.03654).
- Heimersheim, S. (2024). You can remove GPT2’s layernorm by fine-tuning. [arXiv:2409.13710](https://arxiv.org/abs/2409.13710).
- Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., & Steinhardt, J. (2020). Measuring massive multitask language understanding. [arXiv:2009.03300](https://arxiv.org/abs/2009.03300).
- Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., & Steinhardt, J. (2021). Measuring mathematical problem solving with the math dataset. [arXiv:2103.03874](https://arxiv.org/abs/2103.03874).
- Hermann, K. M., Kocisky, T., Grefenstette, E., Espeholt, L., Kay, W., Suleyman, M., & Blunsom, P. (2015). Teaching machines to read and comprehend. *Advances in Neural Information Processing Systems*, 28, 1693–1701.
- Ho, N., Schmid, L., & Yun, S.-Y. (2022). Large language models are reasoning teachers. [arXiv:2212.10071](https://arxiv.org/abs/2212.10071).
- Hsieh, C.-P., Sun, S., Kriman, S., Acharya, S., Rekes, D., Jia, F., Zhang, Y., & Ginsburg, B. (2024). Ruler: What’s the real context size of your long-context language models? [arXiv:2404.06654](https://arxiv.org/abs/2404.06654).

- Hsieh, C.-Y., Li, C.-L., Yeh, C.-k., Nakhost, H., Fujii, Y., Ratner, A., Krishna, R., Lee, C.-Y., & Pfister, T. (2023). Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes. In A. Rogers, J. Boyd-Graber, & N. Okazaki (Eds.), *Findings of the association for computational linguistics: ACL 2023* (pp. 8003–8017). Toronto, Canada: Association for Computational Linguistics. <https://doi.org/10.18653/v1/2023.findings-acl.507>.
- Hsu, Y.-C., Hua, T., Chang, S., Lou, Q., Shen, Y., & Jin, H. (2022). Language model compression with weighted low-rank factorization. In *International conference on learning representations* (p. N/A).
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W. et al. (2022). Lora: Low-rank adaptation of large language models. *ICLR*, 1(2), 3.
- Hu, H., Zhao, P., Li, P., Zheng, Y., Wang, Z., & Yuan, X. (2025a). Fasp: Fast and accurate structured pruning of large language models. [arXiv:2501.09412](https://arxiv.org/abs/2501.09412).
- Hu, X., Cheng, Y., Yang, D., Chen, Z., Xu, Z., Jiangyong, Y., Xu, C., Yuan, Z., Jiang, Z., & Zhou, S. (2025b). OSTQuant: Refining large language model quantization with orthogonal and scaling transformations for better distribution fitting. In *The thirteenth international conference on learning representations*. <https://openreview.net/forum?id=rAcgDBdKnP>.
- Huang, X., Huang, Y.-L., & Wen, Z. (2025). SoLA: Leveraging soft activation sparsity and low-rank decomposition for large language model compression. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 17494–17502). (vol. 39).
- Huang, Y., Chen, Y., Yu, Z., & McKeown, K. (2022). In-context learning distillation: Transferring few-shot learning ability of pre-trained language models. [arXiv:2212.10670](https://arxiv.org/abs/2212.10670).
- Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning* (pp. 448–456). pmlr.
- Javaheripi, M., Bubeck, S., Abdin, M., Aneja, J., Bubeck, S., Mendes, C. C. T., Chen, W., Del Giorno, A., Eldan, R., Gopi, S. et al. (2023). Phi-2: The surprising power of small language models. *Microsoft Research Blog*, 1(3), 3.
- Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., de las Casas, D., Bressand, F., Lengyel, G., Lample, G., Saulnier, L., Lavaud, L. R., Lachaux, M.-A., Stock, P., Scao, T. L., Lavril, T., Wang, T., Lacroix, T., & Sayed, W. E. (2023a). Mistral 7b. [arXiv:2310.06825](https://arxiv.org/abs/2310.06825).
- Jiang, A. Q., Sablayrolles, A., Rous, A., Mensch, A., Savary, B., Bamford, C., Chaplot, D. S., de las, C. D., Hanna, E. B., Bressand, F. et al. (2024). Mixtral of experts. [arXiv:2401.04088](https://arxiv.org/abs/2401.04088).
- Jiang, Y., Chan, C., Chen, M., & Wang, W. (2023b). Lion: Adversarial distillation of proprietary large language models. In *The 2023 conference on empirical methods in natural language processing*. <https://openreview.net/forum?id=nucyYJZS5z>.
- Joshi, M., Choi, E., Weld, D. S., & Zettlemoyer, L. (2017). Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. [arXiv:1705.03551](https://arxiv.org/abs/1705.03551).
- Kamradt, G. (2023). Needle in a haystack - pressure testing LLMs. Github. URL <https://github.com/gkamradt/LLMTestNeedleInAHaystack/tree/main>.
- Katharopoulos, A., Vyas, A., Pappas, N., & Fleuret, F. (2020). Transformers are rnns: Fast autoregressive transformers with linear attention. In *International conference on machine learning* (pp. 5156–5165). PMLR.
- Kaushal, A., Vaidhya, T., & Rish, I. (2024). LoRD: Low-rank decomposition of monolingual code LLMs for one-shot compression. In *ICML 2024 workshop on foundation models in the wild*. <https://openreview.net/forum?id=br49PQvUmp>.
- Kim, M., Lee, S., Lee, J., Hong, S., Chang, D.-S., Sung, W., & Choi, J. (2023). Token-scaled logit distillation for ternary weight generative language models. *Advances in Neural Information Processing Systems*, 36, 42097–42118.
- Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A. A., Milan, K., Quan, J., Ramalho, T., Grabska-Barwinska, A. et al. (2017). Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13), 3521–3526.
- Ko, J., Kim, S., Chen, T., & Yun, S.-Y. (2024). DistiLLM: Towards streamlined distillation for large language models. In *Forty-first international conference on machine learning*. <https://openreview.net/forum?id=lsHZNNoC7r>.
- Koca, N. A., Do, A. T., & Chang, C.-H. (2023). Hardware-efficient softmax approximation for self-attention networks. In *2023 IEEE International symposium on circuits and systems (ISCAS)* (pp. 1–5). IEEE.
- Lan, Z., Chen, M., Goodman, S., Gimpel, K., Sharma, P., & Soricut, R. (2019). Albert: A lite bert for self-supervised learning of language representations. [arXiv:1909.11942](https://arxiv.org/abs/1909.11942).
- Lang, J., Guo, Z., & Huang, S. (2024). A comprehensive study on quantization techniques for large language models. In *2024 4th international conference on artificial intelligence, robotics, and communication (ICAIRC)* (pp. 224–231). IEEE.
- Le, Q., Diao, E., Wang, Z., Wang, X., Ding, J., Yang, L., & Anwar, A. (2025). Probe pruning: Accelerating LLMs through dynamic pruning via model-probing. In Y. Yue, A. Garg, N. Peng, F. Sha, & R. Yu (Eds.), *International conference on representation learning* (pp. 72494–72513). (vol. 2025). https://proceedings.iclr.cc/paper_files/paper/2025/file/b44efaa3b7a389ea56c7ee9d6589e800-Paper-Conference.pdf.
- Le Scao, T., Fan, A., Akiki, C., Pavlick, E., Ilıc, S., Hesslow, D., Castagné, R., Luccioni, A. S., Yvon, F., Gallé, M. et al. (2023). Bloom: A 176b-parameter open-access multilingual language model. [arXiv:2211.05100](https://arxiv.org/abs/2211.05100).
- Lee, D., Lee, J.-Y., Zhang, G., Tiwari, M., & Mirhoseini, A. (2024). Cats: Contextually-aware thresholding for sparsity in large language models. [arXiv:2404.08763](https://arxiv.org/abs/2404.08763).
- Lefauveux, B., Massa, F., Liskovich, D., Xiong, W., Caggiano, V., Naren, S., Xu, M., Hu, J., Tintore, M., Zhang, S., Labatut, P., Haziza, D., Wehrstedt, L., Reizenstein, J., & Sizov, G. (2022). xformers: A modular and hackable transformer modelling library. <https://github.com/facebookresearch/xformers>.
- Lepikhin, D., Lee, H., Xu, Y., Chen, D., Firat, O., Huang, Y., Krikun, M., Shazeer, N., & Chen, Z. (2021). {GS}hard: Scaling giant models with conditional computation and automatic sharding. In *International conference on learning representations*. <https://openreview.net/forum?id=qwr7XHTmYb>.
- Lewis, M., Bhosale, S., Dettmers, T., Goyal, N., & Zettlemoyer, L. (2021). Base layers: Simplifying training of large, sparse models. In *International conference on machine learning* (pp. 6265–6274). PMLR.
- Lewis, M., Liu, Y., Goyal, N., Ghazvininejad, M., Mohamed, A., Levy, O., Stoyanov, V., & Zettlemoyer, L. (2020). BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In D. Jurafsky, J. Chai, N. Schluter, & J. Tetraault (Eds.), *Proceedings of the 58th annual meeting of the association for computational linguistics* (pp. 7871–7880). Online: Association for Computational Linguistics. <https://doi.org/10.18653/v1/2020.acl-main.703>.
- Li, P., Yin, L., & Liu, S. (2024). Mix-In: Unleashing the power of deeper layers by combining pre-In and post-In. [arXiv:2412.13795](https://arxiv.org/abs/2412.13795).
- Liang, C., Zuo, S., Zhang, Q., He, P., Chen, W., & Zhao, T. (2023). Less is more: Task-aware layer-wise distillation for language model compression. In *International conference on machine learning* (pp. 20852–20867). PMLR.
- Liang, X., Wang, H., Lai, H., Niu, S., Song, S., Yang, J., Zhao, J., Xiong, F., Tang, B., & Li, Z. (2025). Seap: Training-free sparse expert activation pruning unlock the brainpower of large language models. [arXiv:2503.07605](https://arxiv.org/abs/2503.07605).
- Lin, C.-Y. (2004). Rouge: A package for automatic evaluation of summaries. In *Text summarization branches out* (pp. 74–81).
- Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.-M., Wang, W.-C., Xiao, G., Dang, X., Gan, C., & Han, S. (2024). Awq: Activation-aware weight quantization for on-device llm compression and acceleration. In *MLsys*. https://proceedings.mlsys.org/paper_files/paper/2024/hash/42a452cbafa9dd64e9ba4aa95cc1ef21-Abstract-Conference.html.
- Liu, A., Feng, B., Xue, B., Wang, B., Wu, B., Lu, C., Zhao, C., Deng, C., Zhang, C., Ruan, C. et al. (2024a). Deepseek-v3 technical report. [arXiv:2412.19437](https://arxiv.org/abs/2412.19437).
- Liu, J., Gong, R., Wei, X., Dong, Z., Cai, J., & Zhuang, B. (2023). Qllm: Accurate and efficient low-bitwidth quantization for large language models. [arXiv:2310.08041](https://arxiv.org/abs/2310.08041).
- Liu, L., Zhang, S., Kuang, Z., Zhou, A., Xue, J.-H., Wang, X., Chen, Y., Yang, W., Liao, Q., & Zhang, W. (2021). Group fisher pruning for practical neural network compression. In *International conference on machine learning* (pp. 7021–7032). PMLR.
- Liu, S., He, B., Wu, H., & Song, L. (2025a). Optishear: Towards efficient and adaptive pruning of large language models via evolutionary optimization. [arXiv:2502.10735](https://arxiv.org/abs/2502.10735).
- Liu, S., Tao, G., Zou, Y., Chow, D., Fan, Z., Lei, K., Pan, B., Sylvester, D., Kieliang, G., & Saligane, M. (2024b). Consmax: Hardware-friendly alternative softmax with learnable parameters. In *Proceedings of the 43rd IEEE/ACM international conference on computer-aided design* (pp. 1–9).
- Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., & Stoyanov, V. (2019). Roberta: A robustly optimized bert pretraining approach. [arXiv:1907.11692](https://arxiv.org/abs/1907.11692).
- Liu, Z., Oguz, B., Zhao, C., Chang, E., Stock, P., Mehdad, Y., Shi, Y., Krishnamoorthi, R., & Chandra, V. (2024c). LLM-QAT: Data-free quantization aware training for large language models. In L.-W. Ku, A. Martins, & V. Srikumar (Eds.), *Findings of the association for computational linguistics: ACL 2024* (pp. 467–484). Bangkok, Thailand: Association for Computational Linguistics. <https://doi.org/10.18653/v1/2024.findings-acl.26>.
- Liu, Z., Zhao, C., Fedorov, I., Soran, B., Choudhary, D., Krishnamoorthi, R., Chandra, V., Tian, Y., & Blakevoort, T. (2025b). Spinquant: LLM quantization with learned rotations. In *The thirteenth international conference on learning representations*. <https://openreview.net/forum?id=ogO6DGE6FZ>.
- Liu, Z., Zhao, C., Iandola, F., Lai, C., Tian, Y., Fedorov, I., Xiong, Y., Chang, E., Shi, Y., Krishnamoorthi, R., Lai, L., & Chandra, V. (2024d). MobileLLM: Optimizing sub-billion parameter language models for on-device use cases. [arXiv:2402.14905](https://arxiv.org/abs/2402.14905).
- Loshchilov, I., Hsieh, C.-P., Sun, S., & Ginsburg, B. (2024). ngpt: Normalized transformer with representation learning on the hypersphere. [arXiv:2410.01131](https://arxiv.org/abs/2410.01131).
- Lupart, S., Allaniejadi, M., & Kanoulas, E. (2025). DiSCo: LLM knowledge distillation for efficient sparse retrieval in conversational search. In *Proceedings of the 48th international ACM SIGIR conference on research and development in information retrieval SIGIR '25* (p. 9–19). New York, NY, USA: Association for Computing Machinery. <https://doi.org/10.1145/3726302.3729966>.
- Ma, S., Wang, H., Ma, L., Wang, L., Wang, W., Huang, S., Dong, L., Wang, R., Xue, J., & Wei, F. (2024). The era of 1-bit llms: All large language models are in 1.58 bits. [arXiv:2402.17764](https://arxiv.org/abs/2402.17764), 1.
- Ma, X., Fang, G., & Wang, X. (2023). Llm-pruner: On the structural pruning of large language models. *Advances in Neural Information Processing Systems*, 36, 21702–21720.
- Ma, X., Kong, X., Wang, S., Zhou, C., May, J., Ma, H., & Zettlemoyer, L. (2021). Luna: Linear unified nested attention. *Advances in Neural Information Processing Systems*, 34, 2441–2453.
- Mahoney, M. (2011). Large text compression benchmark.
- Marcus, M., Santorini, B., & Marcinkiewicz, M. A. (1993). Building a large annotated corpus of english: The penn treebank. *Computational Linguistics*, 19(2), 313–330.
- Merity, S., Xiong, C., Bradbury, J., & Socher, R. (2016a). Pointer sentinel mixture models. [arXiv:1609.07843](https://arxiv.org/abs/1609.07843).
- Merity, S., Xiong, C., Bradbury, J., & Socher, R. (2016b). Pointer sentinel mixture models. [arXiv:1609.07843](https://arxiv.org/abs/1609.07843).
- Milakov, M., & Gimpel, N. (2018). Online normalizer calculation for softmax. [arXiv:1805.02867](https://arxiv.org/abs/1805.02867).
- Nguyen, T. Q., & Salazar, J. (2019). Transformers without tears: Improving the normalization of self-attention. In J. Niehues, R. Cattoni, S. Stüker, M. Negri, M. Turchi, T.-L. Ha, E. Salesky, R. Sanabria, L. Barrault, L. Specia, & M. Federico (Eds.), *Proceedings of the 16th international conference on spoken language translation*. Hong Kong: Association for Computational Linguistics. <https://aclanthology.org/2019.iwslt-1.17/>.
- Ning, X., Lin, Z., Zhou, Z., Wang, Z., Yang, H., & Wang, Y. (2024). Skeleton-of-thought: Prompting LLMs for efficient parallel generation. In *The twelfth international conference on learning representations*. <https://openreview.net/forum?id=mqVgBbNCm9>.

- OpenOrca Contributors (2023). Openorca: Open-source instruction-following dataset. <https://huggingface.co/datasets/Open-Orca/OpenOrca>. Hugging Face Dataset card; arXiv works associated include [arXiv:2306.02707](https://arxiv.org/abs/2306.02707), [arXiv:2301.13688](https://arxiv.org/abs/2301.13688), [arXiv:2302.13971](https://arxiv.org/abs/2302.13971).
- Pagliardini, M., Paliotto, D., Jaggi, M., & Fleuret, F. (2023). Faster causal attention over large sequences through sparse flash attention. [arXiv:2306.01160](https://arxiv.org/abs/2306.01160).
- Park, Y., Budhathoki, K., Chen, L., Kübler, J. M., Huang, J., Kleindessner, M., Huan, J., Cevher, V., Wang, Y., & Karypis, G. (2024). Inference optimization of foundation models on ai accelerators. In *Proceedings of the 30th ACM SIGKDD conference on knowledge discovery and data mining* (pp. 6605–6615).
- Peng, B., Quesnelle, J., Fan, H., & Shippole, E. (2024). YaRN: Efficient context window extension of large language models. In *The twelfth international conference on learning representations*. <https://openreview.net/forum?id=wHbfxhZu1u>.
- Peng, H., Pappas, N., Yogatama, D., Schwartz, R., Smith, N. A., & Kong, L. (2021). Random feature attention. [arXiv:2103.02143](https://arxiv.org/abs/2103.02143).
- Popović, M. (2015). ChrF: Character n-gram f-score for automatic MT evaluation. In *Proceedings of the tenth workshop on statistical machine translation* (pp. 392–395).
- Rae, J. W., Potapenko, A., Jayakumar, S. M., & Lillcrap, T. P. (2019). Compressive transformers for long-range sequence modelling. [arXiv:1911.05507](https://arxiv.org/abs/1911.05507).
- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., & Liu, P. J. (2020a). Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140), 1–67.
- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., & Liu, P. J. (2020b). Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140), 1–67. <http://jmlr.org/papers/v21/02-074.html>.
- Rajpurkar, P., Zhang, J., Lopyrev, K., & Liang, P. (2016). Squad: 100,000+ Questions for machine comprehension of text. [arXiv:1606.05250](https://arxiv.org/abs/1606.05250).
- Reid, M., Marrese-Taylor, E., & Matsuo, Y. (2021). Subformer: Exploring weight sharing for parameter efficiency in generative transformers. [arXiv:2101.00234](https://arxiv.org/abs/2101.00234).
- Sakaguchi, K., Bras, R. L., Bhagavatula, C., & Choi, Y. (2021). Winogrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 64(9), 99–106.
- Sap, M., Rashkin, H., Chen, D., Le Bras, R., & Choi, Y. (2019). Social IQa: Commonsense reasoning about social interactions. In K. Inui, J. Jiang, V. Ng, & X. Wan (Eds.), *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)* (pp. 4463–4473). Hong Kong, China: Association for Computational Linguistics. <https://doi.org/10.18653/v1/D19-1454>.
- Sharma, L., Graesser, L., Nangia, N., & Evcı, U. (2019). Natural language understanding with the quora question pairs dataset. [arXiv:1907.01041](https://arxiv.org/abs/1907.01041).
- Shaw, P., Uszkoreit, J., & Vaswani, A. (2018). Self-attention with relative position representations. In *Proceedings of the 2018 conference of the north american chapter of the association for computational linguistics: Human language technologies, volume 2 (short papers)* (pp. 464–468).
- Shazeer, N. (1911). Fast transformer decoding: One write-head is all you need, 2019. <https://arxiv.org/abs/1911.08026>.
- Shazeer, N. (2020). Glu variants improve transformer. [arXiv:2002.05202](https://arxiv.org/abs/2002.05202).
- Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., & Dean, J. (2017). Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International conference on learning representations*. <https://openreview.net/forum?id=B1ckMDqlg>.
- Slivkins, A. et al. (2019). Introduction to multi-armed bandits. *Foundations and Trends® in Machine Learning*, 12(1–2), 1–286.
- Song, C., Han, X., Zhang, Z., Hu, S., Shi, X., Li, K., Chen, C., Liu, Z., Li, G., Yang, T., & Sun, M. (2025). ProSparse: Introducing and enhancing intrinsic activation sparsity within large language models. In O. Rambow, L. Wanner, M. Apidianaki, H. Al-Khalifa, B. D. Eugenio, & S. Schockaert (Eds.), *Proceedings of the 31st international conference on computational linguistics* (pp. 2626–2644). Abu Dhabi, UAE: Association for Computational Linguistics. <https://aclanthology.org/2025.coling-main.180/>.
- Stevens, J. R., Venkatesan, R., Dai, S., Khailany, B., & Raghunathan, A. (2021). Softmax: Hardware/software co-design of an efficient softmax for transformers. In *2021 58th ACM/IEEE design automation conference (DAC)* (pp. 469–474). IEEE.
- Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., & Liu, Y. (2024). Reformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568, 127063. <https://www.sciencedirect.com/science/article/pii/S0925231223011864>. <https://doi.org/10.1016/j.neucom.2023.127063>.
- Sun, M., Liu, Z., Bair, A., & Kolter, J. Z. (2024a). A simple and effective pruning approach for large language models. In *The twelfth international conference on learning representations*. <https://openreview.net/forum?id=PxoFut3dWW>.
- Sun, M., Liu, Z., Bair, A., & Kolter, J. Z. (2024b). A simple and effective pruning approach for large language models. [arXiv:2306.11695](https://arxiv.org/abs/2306.11695).
- Sun, Z., Yu, H., Song, X., Liu, R., Yang, Y., & Zhou, D. (2020). MobileBERT: A compact task-agnostic BERT for resource-limited devices. In D. Jurafsky, J. Chai, N. Schluter, & J. Tetreault (Eds.), *Proceedings of the 58th annual meeting of the association for computational linguistics* (pp. 2158–2170). Online: Association for Computational Linguistics. <https://doi.org/10.18653/v1/2020.acl-main.195>.
- Talmor, A., Herzig, J., Lourie, N., & Berant, J. (2018). Commonsenseqa: A question answering challenge targeting commonsense knowledge. [arXiv:1811.00937](https://arxiv.org/abs/1811.00937).
- Tay, Y., Dehghani, M., Abnar, S., Shen, Y., Bahri, D., Pham, P., Rao, J., Yang, L., Ruder, S., & Metzler, D. (2020). Long range arena: A benchmark for efficient transformers. [arXiv:2011.04006](https://arxiv.org/abs/2011.04006).
- Tay, Y., Dehghani, M., Bahri, D., & Metzler, D. (2022). Efficient transformers: A survey. *ACM Computing Surveys*, 55(6). <https://doi.org/10.1145/3530811>.
- Thawakar, O., Vayani, A., Khan, S., Cholakkal, H., Anwer, R. M., Felsberg, M., Baldwin, T., Xing, E. P., & Khan, F. S. (2024). Mobillama: Towards accurate and lightweight fully transparent gpt. [arXiv:2402.16840](https://arxiv.org/abs/2402.16840).
- Timiryasov, I., & Tastet, J.-L. (2023). Baby llama: Knowledge distillation from an ensemble of teachers trained on a small dataset with no performance penalty. In A. Warstadt, A. Mueller, L. Choshen, E. Wilcox, C. Zhuang, J. Ciro, R. Mosquera, B. Paranjabe, A. Williams, T. Linzen, & R. Cotterell (Eds.), *Proceedings of the babyLM challenge at the 27th conference on computational natural language learning* (pp. 279–289). Singapore: Association for Computational Linguistics. <https://doi.org/10.18653/v1/2023.conll-babylm.24>.
- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F. et al. (2023a). Llama: Open and efficient foundation language models. [arXiv:2302.13971](https://arxiv.org/abs/2302.13971).
- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S. et al. (2023b). Llama 2: Open foundation and fine-tuned chat models. [arXiv:2307.09288](https://arxiv.org/abs/2307.09288).
- Tseng, A., Chee, J., Sun, Q., Kuleshov, V., & De Sa, C. (2024). QulP#: Even better LLM quantization with hadamard incoherence and lattice codebooks. In *Forty-first international conference on machine learning*. <https://openreview.net/forum?id=9BrydUVcoe>.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017a). Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 6000–6010.
- Vicuna (2023). Vicuna: An open-source chatbot impressing GPT-4 with 90% ChatGPT quality. <https://lmsys.org/blog/2023-03-30-vicuna/>.
- Voita, E., Talbot, D., Moiseev, F., Sennrich, R., & Titov, I. (2019). Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned. In A. Korhonen, D. Traum, & L. Marquez (Eds.), *Proceedings of the 57th annual meeting of the association for computational linguistics* (pp. 5797–5808). Florence, Italy: Association for Computational Linguistics. <https://doi.org/10.18653/v1/P19-1580>.
- Wan, Z., Wang, X., Liu, C., Alam, S., Zheng, Y., Liu, J., Qu, Z., Yan, S., Zhu, Y., Zhang, Q., Chowdhury, M., & Zhang, M. (2024). Efficient large language models: A survey. *Transactions on Machine Learning Research*, Survey Certification <https://openreview.net/forum?id=bsCCJHbO8A>.
- Wang, A., Singh, A., Michael, J., Hill, F., Levy, O., & Bowman, S., et al. (2018). GLUE: A multi-task benchmark and analysis platform for natural language understanding. In T. Linzen, G. Chrupala, & A. Alishahi (Eds.), *Proceedings of the 2018 EMNLP workshop blackboxNLP: analyzing and interpreting neural networks for NLP* (pp. 353–355). Brussels, Belgium: Association for Computational Linguistics. <https://doi.org/10.18653/v1/W18-5446>.
- Wang, H., Ma, S., Dong, L., Huang, S., Wang, H., Ma, L., Yang, F., Wang, R., Wu, Y., & Wei, F. (2023). Bitnet: Scaling 1-bit transformers for large language models. [arXiv:2310.11453](https://arxiv.org/abs/2310.11453).
- Wang, J., Chen, Y.-G., Lin, I.-C., Li, B., & Zhang, G. L. (2024). Basis sharing: Cross-layer parameter sharing for large language model compression. [arXiv:2410.03765](https://arxiv.org/abs/2410.03765).
- Wang, S., Li, B. Z., Khabsa, M., Fang, H., & Ma, H. (2020). Linformer: Self-attention with linear complexity. [arXiv:2006.04768](https://arxiv.org/abs/2006.04768).
- Wang, X., Alam, S., Wan, Z., Shen, H., & Zhang, M. (2025a). SVD-LLM v2: Optimizing singular value truncation for large language model compression. In L. Chiruzzo, A. Ritter, & L. Wang (Eds.), *Proceedings of the 2025 conference of the nations of the americas chapter of the association for computational linguistics: Human language technologies (volume 1: Long papers)*.
- Wang, X., Zheng, Y., Wan, Z., & Zhang, M. (2025b). SVD-LLM: Truncation-aware singular value decomposition for large language model compression. In *The thirteenth international conference on learning representations*.
- Wei, X., Zhang, Y., Li, Y., Zhang, X., Gong, R., Guo, J., & Liu, X. (2023). Outlier suppression+: Accurate quantization of large language models by equivalent and effective shifting and scaling. In H. Bouamor, J. Pino, & K. Bali (Eds.), *Proceedings of the 2023 conference on empirical methods in natural language processing* (pp. 1648–1665). Singapore: Association for Computational Linguistics. <https://doi.org/10.18653/v1/2023.emnlp-main.102>.
- Welbl, J., Stenetorp, P., & Riedel, S. (2018). Constructing datasets for multi-hop reading comprehension across documents. *Transactions of the Association for Computational Linguistics*, 6, 287–302.
- Xiao, G., Lin, J., Seznec, M., Wu, H., Demouth, J., & Han, S. (2023). Smoothquant: Accurate and efficient post-training quantization for large language models. In *International conference on machine learning* (pp. 38087–38099). PMLR.
- Xiong, R., Yang, Y., He, D., Zheng, K., Zheng, S., Xing, C., Zhang, H., Lan, Y., Wang, L., & Liu, T. (2020). On layer normalization in the transformer architecture. In *International conference on machine learning* (pp. 10524–10533). PMLR.
- Xiong, Y., Zeng, Z., Chakraborty, R., Tan, M., Fung, G., Li, Y., & Singh, V. (2021). Nyströmformer: A nyström-based algorithm for approximating self-attention. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 14138–14148). (vol. 35).
- Xu, J., Sun, X., Zhang, Z., Zhao, G., & Lin, J. (2019). Understanding and improving layer normalization. *Advances in Neural Information Processing Systems*, 32, 4381–4391.
- Xu, W., Han, R., Wang, Z., Le, L., Madeka, D., Li, L., Wang, W. Y., Agarwal, R., Lee, C.-Y., & Pfister, T. (2025). Speculative knowledge distillation: Bridging the teacher-student gap through interleaved sampling. In *The thirteenth international conference on learning representations*. <https://openreview.net/forum?id=EgJhwYR2tB>.
- Xu, X., Li, M., Tao, C., Shen, T., Cheng, R., Li, J., Xu, C., Tao, D., & Zhou, T. (2024a). A survey on knowledge distillation of large language models. *CoRR*, [arXiv:2402.13116](https://arxiv.org/abs/2402.13116).
- Xu, Y., Han, X., Yang, Z., Wang, S., Zhu, Q., Liu, Z., Liu, W., & Che, W. (2024b). Onebit: Towards extremely low-bit large language models. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, & C. Zhang (Eds.), *Advances in neural information processing systems* (pp. 66357–66382). Curran Associates, Inc. (37). https://proceedings.neurips.cc/paper_files/paper/2024/file/7a7a3f53faaf0161be0fcb57e5fa078-Paper-Conference.pdf. <https://doi.org/10.52202/079017-2122>.
- Xue, L., Constant, N., Roberts, A., Kale, M., Al-Rfou, R., Siddhant, A., Barua, A., & Raf-fel, C. (2021). mT5: A massively multilingual pre-trained text-to-text transformer. In K. Toutanova, A. Rumshisky, L. Zettlemoyer, D. Hakkani-Tur, I. Beltagy, S. Bethard,

- R. Cotterell, T. Chakraborty, & Y. Zhou (Eds.), *Proceedings of the 2021 conference of the north american chapter of the association for computational linguistics: Human language technologies* (pp. 483–498). Online: Association for Computational Linguistics. <https://doi.org/10.18653/v1/2021.naacl-main.41>.
- Yao, J., & Gultepe, E. (2026). Spq: An ensemble technique for large language model compression. *arXiv:2602.18420*.
- Yuan, Z., Shang, Y., Song, Y., Wu, Q., Yan, Y., & Sun, G. (2023). Asvd: Activation-aware singular value decomposition for compressing large language models. *arXiv:2312.05821*.
- Zellers, R., Holtzman, A., Bisk, Y., Farhadi, A., & Choi, Y. (2019). Hellaswag: Can a machine really finish your sentence? *arXiv:1905.07830*.
- Zhang, B., & Sennrich, R. (2019). Root mean square layer normalization. *Advances in Neural Information Processing Systems*, 32, 12360–12371.
- Zhang, J., Wang, H., Jiang, K., Yang, S., Zheng, K., Xi, H., Wang, Z., Zhu, H., Zhao, M., Stoica, I. et al. (2025a). Sla: Beyond sparsity in diffusion transformers via fine-tunable sparse-linear attention. *arXiv:2509.24006*.
- Zhang, M., Arora, S., Chalamala, R., Spector, B. F., Wu, A., Ramesh, K., Singhal, A., & Re, C. (2025b). LoLCATs: On low-rank linearizing of large language models. In *The thirteenth international conference on learning representations*.
- Zhang, S., Roller, S., Goyal, N., Artetxe, M., Chen, M., Chen, S., Dewan, C., Diab, M., Li, X., Lin, X. V. et al. (2022a). Opt: Open pre-trained transformer language models. *arXiv:2205.01068*.
- Zhang, S., Zhang, X., Sun, Z., Chen, Y., & Xu, J. (2024). Dual-space knowledge distillation for large language models. In Y. Al-Onaizan, M. Bansal, & Y.-N. Chen (Eds.), *Proceedings of the 2024 conference on empirical methods in natural language processing* (pp. 18164–18181). Miami, Florida, USA: Association for Computational Linguistics. <https://doi.org/10.18653/v1/2024.emnlp-main.1010>.
- Zhang, T., & Shrivastava, A. (2025). Leanquant: Accurate and scalable large language model quantization with loss-error-aware grid. In *The thirteenth international conference on learning representations*. <https://openreview.net/forum?id=ISqx8giekS>.
- Zhang, Z., Lin, Y., Liu, Z., Li, P., Sun, M., & Zhou, J. (2022b). MoEification: Transformer feed-forward layers are mixtures of experts. In S. Muresan, P. Nakov, & A. Villavicencio (Eds.), *Findings of the association for computational linguistics: ACL 2022* (pp. 877–890). Dublin, Ireland: Association for Computational Linguistics. <https://doi.org/10.18653/v1/2022.findings-acl.71>.
- Zhang, Z., Liu, Z., Tian, Y., Khaitan, H., Wang, Z., & Li, S. (2025c). R-sparse: Rank-aware activation sparsity for efficient llm inference. *arXiv:2504.19449*.
- Zhou, Z., Ning, X., Hong, K., Fu, T., Xu, J., Li, S., Lou, Y., Wang, L., Yuan, Z., Li, X., Yan, S., Dai, G., Zhang, X.-P., Dong, Y., & Wang, Y. (2024). A survey on efficient inference for large language models. *CoRR*, *arXiv:2404.14294*.
- Zhu, B., Frick, E., Wu, T., Zhu, H., Ganesan, K., Chiang, W.-L., Zhang, J., & Jiao, J. (2024). Starling-7b: Improving helpfulness and harmlessness with RLAI. In *First conference on language modeling*. <https://openreview.net/forum?id=GqDntYTTbk>.
- Zhu, C., Liu, Y., Mei, J., & Zeng, M. (2021). Mediasum: A large-scale media interview dataset for dialogue summarization. *arXiv:2103.06410*.
- Zhu, J., Chen, X., He, K., LeCun, Y., & Liu, Z. (2025). Transformers without normalization. In *Proceedings of the computer vision and pattern recognition conference* (pp. 14901–14911).
- Zhu, W., Hessel, J., Awadalla, A., Gadre, S. Y., Dodge, J., Fang, A., Yu, Y., Schmidt, L., Wang, W. Y., & Choi, Y. (2023). Multimodal c4: An open, billion-scale corpus of images interleaved with text. *Advances in Neural Information Processing Systems*, 36, 8958–8974.



Gilhyeon Lee (Graduate Student Member, IEEE) received the BS degree in Electrical and Information Engineering from Seoul National University of Science and Technology, Seoul, South Korea, in 2024, where he is currently pursuing the MS degree in Electrical and Information Engineering. His research interests include large language models, model compression, processing-in-memory (PIM) architectures, and AI accelerator design.



Seonggeun Kim (Graduate Student Member, IEEE) received the BS degree in Electrical and Information Engineering from Seoul National University of Science and Technology, Seoul, South Korea, in 2024, where he is currently pursuing the MS degree in Electrical and Information Engineering. His research interests include inference optimization in large language models and AI accelerator design.



Dongjun Lee (Graduate Student Member, IEEE) received his BS degree in Electrical and Information Engineering from Seoul National University of Science and Technology (SeoulTech), Seoul, Korea, in 2025. He is currently pursuing the MS degree in the same department at SeoulTech. His research interests include digital system design for deep neural networks, computer architecture, and hardware-aware quantization for edge AI accelerators.



Kyungmin Goh (Graduate Student Member, IEEE) received the BS degree in Electrical and Information Engineering from Seoul National University of Science and Technology, Seoul, South Korea, in 2025, where he is currently pursuing the MS degree in Electrical and Information Engineering. His research interests include large language models, mixture of expert compression, and conditional computation.



Hyun Kim received his BS, MS, and PhD degrees in Electrical Engineering and Computer Science from Seoul National University, Seoul, Korea, in 2009, 2011, and 2015, respectively. From 2015 to 2018, he was with the BK21 Creative Research Engineer Development for IT, Seoul National University, Seoul, Korea, as a BK Assistant Professor. In 2018, he joined the Department of Electrical and Information Engineering, Seoul National University of Science and Technology, Seoul, Korea, where he is currently working as an Associate Professor. His research interests are the areas of algorithms, computer architecture, memory, and SoC design for low-complexity multimedia applications and deep neural networks. His research results have been internationally acclaimed. His research interests are the areas of algorithm, computer architecture, memory system design, and digital system (SoC) design for low-complexity multimedia applications and deep neural networks. Until now, Dr. Kim has published more than 140 papers in international refereed journals and conferences. Dr. Kim also holds more than 32 issued international and national patents. Upon his outstanding academic achievements, he has been the recipient of many honorable awards, including the Haedong Young Engineer Award in 2023, the Outstanding Associate Editor Award of IEEE Transactions on Smart Processing and Computing in 2023, the 2022–2023 IEEE Circuits and Systems Society Design Competition Winner in Asia, Australia, and the Pacific in 2023, the IEEE Seoul Section Best Paper Award in 2022, the IEEE/IEIE ICCE-ASIA Best Paper Award Silver Prize in 2021, and the IEEE/IEIE ICEIC Best Paper Award Silver Prize in 2020.